

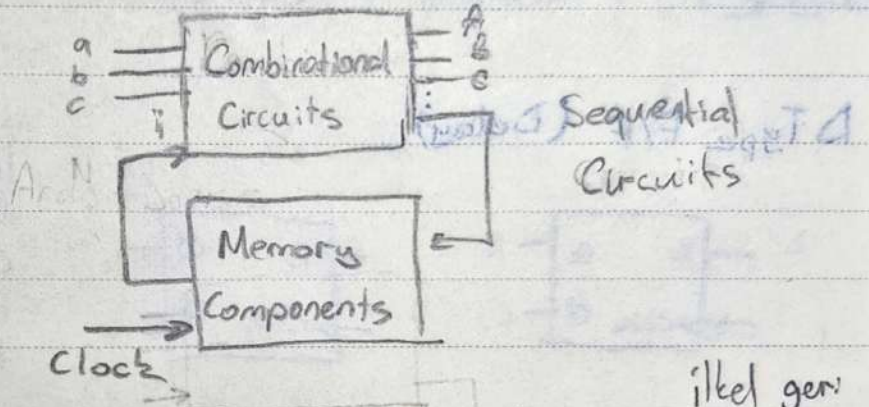
Latches → flip flops → register 3 Depolama birimleri
 ↓ ↓
 asenkron senkron

Ardışıl Devreler:

Ardışıl devrelerde Clock sinyali birlikte bir sonraki duruma geçilir.

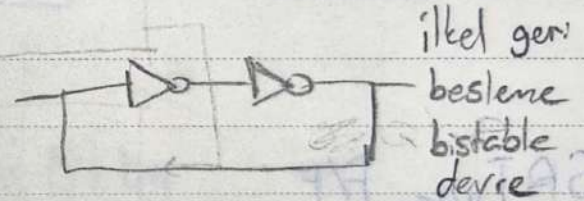
Ardışıl Devreler ikiye ayrılır

- Senkron (Clock sinyali)
- Asenkron



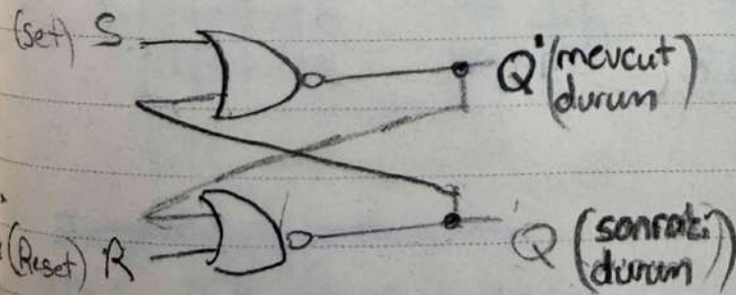
Şu özellikleri içerir:

- * Bellek elemanı
 - * Saat sinyali
 - * Geri Besleme
- Latch



SR Latch

$$P = Q'$$



$$S=1, R=0$$

$$S=0, R=1$$

$$P=0, Q=1$$

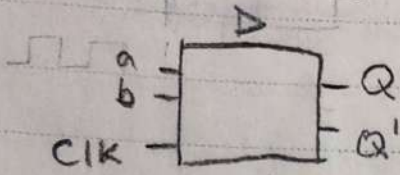
$$P=1, Q=0$$

Store	0
Set	1
reset	0
don't care	x

S	R	q	Q	Q'	özet
0	0	0	0	1	SR Q
0	0	1	1	0	
0	1	0	0	1	
0	1	1	0	1	
1	0	0	1	0	
1	0	1	1	0	1 0 1
1	1	0	x	x	
1	1	1	x	x	

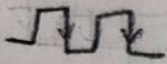
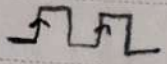
Flip Flops

4 tip Flip-Flop vardır. D, JK, SR, T

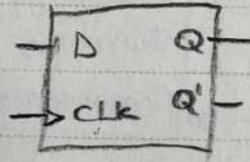
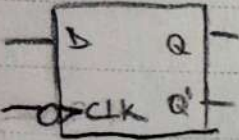


q: mevcut durum

Q: sonraki durum



D Type F/F (Delay)



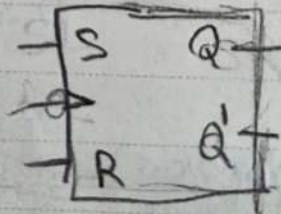
D	q	Q
0	0	0
0	1	0
1	0	1
1	1	1

$\star Q = D$

SR Type F/F

S	R	Q
0	0	q
0	1	0
1	0	1
1	1	-

$\star Q = S + R' \cdot q$



S	R	q	Q
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	-
1	1	1	-

T Type F/F (Toggle)

T	q	Q
0	0	0
0	1	1
1	0	1
1	1	0

1 verilirse anliz durum tersine döner.

$Q = T \oplus q$

T	Q
0	q
1	q'

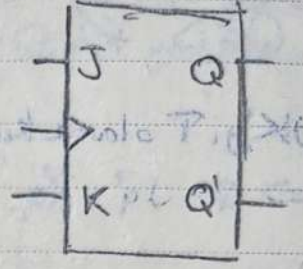
asentronlardır { Clear girişi: çıkışı 0 yapar
 Clk'a göre önceliklidir. preset girişi: çıkışı 1 yapar

JK Type F/F

J	K	q	Q
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

q \ JK	00	01	11	10
0			1	1
1	1			1

$$Q = Jq' + K'q$$



Not: 1 1 durumuna kadar SR sonrasında T gibi davranır.

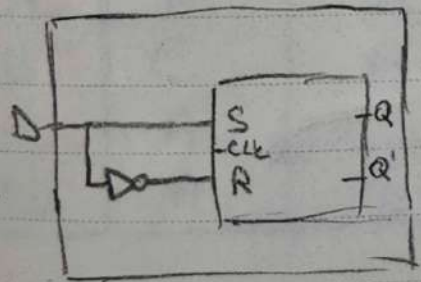
JK	Q
00	q
01	0
10	1
11	q'

Flip Flop dönüşümleri:

SR'yi D olarak kullanma:

$$Q = S + R'q \rightarrow Q = D(1+q) \rightarrow Q = D + Dq$$

S = D
R = D'

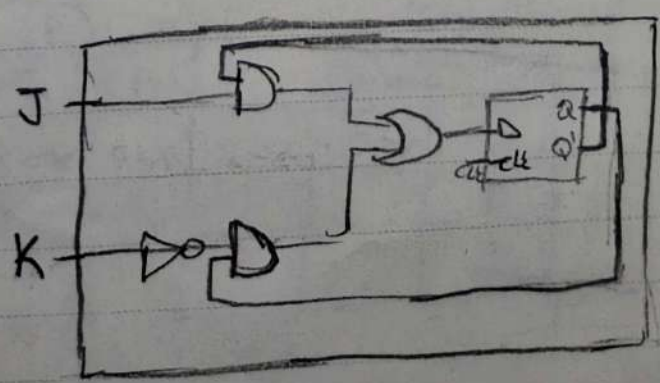


~~JK'yi D olarak kullanma:~~

D'yi JK olarak kullanma:

$$Q = Jq' + K'q$$

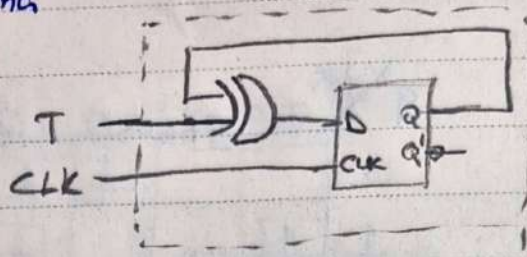
$$Q = D$$



D'yi T olarak kullanma

$$Q = T \oplus q$$

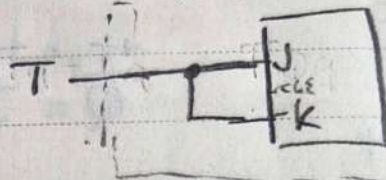
$$Q = D$$



JK'yı T olarak kullanma:

$$JK \Rightarrow Q = Jq' + K'q \quad J = K = T \text{ yapılırsa}$$

$$Q = Tq' + T'q = T \oplus q$$



Excitation Table

D için:

q	Q	D
0	0	0
0	1	1
1	0	0
1	1	1

SR için:

q	Q	SR
0	0	0x
0	1	10
1	0	01
1	1	x0

T için:

q	Q	T
0	0	0
0	1	1
1	0	1
1	1	0

JK için:

q	Q	JK
0	0	0x
0	1	1x
1	0	x1
1	1	x0

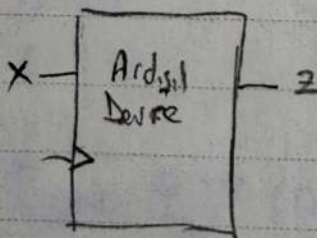
Önceki
bölge

Öncekinin
bölge olarak
istiyorsam D'ye
ne vermem

~~Durum~~

Durum Tablosu

1. Flip Flop



Giris (X)	q	Q	çıkış (Z)
0	0		
0	1		
1	0		
1	1		

(Diken)

(Yatay)

q	Q	2
	x=0	x=1
0	0	1
1	0	1

2 Flip Flop

$q_1 q_2$	$Q_1 Q_2$		z	
	$x=0$	$x=1$	$x=0$	$x=1$
00	00	01	0	0
01	10	00	1	0
10	00	11	0	1
11	00	01	0	0

Burada 00 01 diye yazmak yerine A, B, C, D diye yazılabilir.

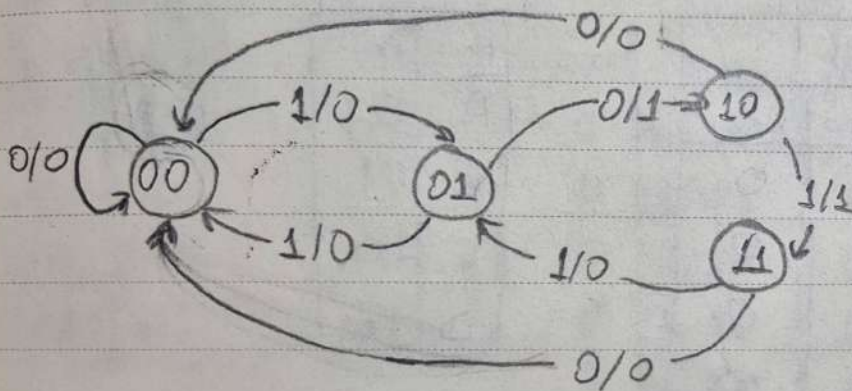
Harf Durum

A=00

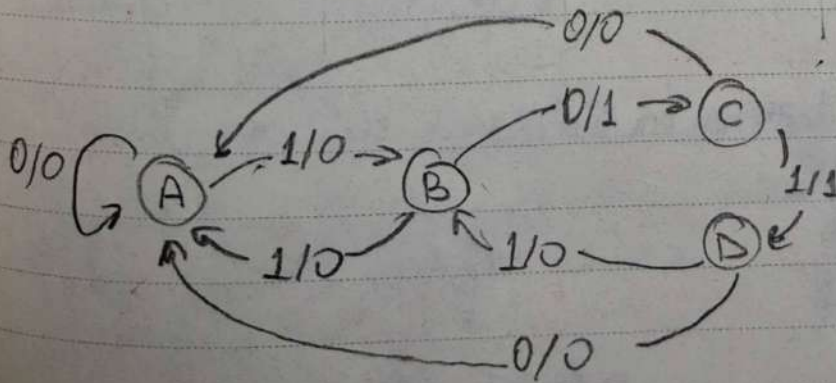
B=01

C=10

D=11



$q_1 q_2$	$Q_1 Q_2$		z	
	$x=0$	$x=1$	$x=0$	$x=1$
A	A	B	0	0
B	C	A	1	0
C	A	D	0	1
D	A	B	0	0



Ardışıl Devre Analizi

Hafta 2 7. sayfa örneği

1. adım

$$Q_1 = x \cdot (q_1 + q_2)$$

$$Q_2 = x \cdot q_1'$$

$$z = x' \cdot (q_1 + q_2)$$

Geçiş
sonraki durum
denklemleri
bulunur.

2. adım

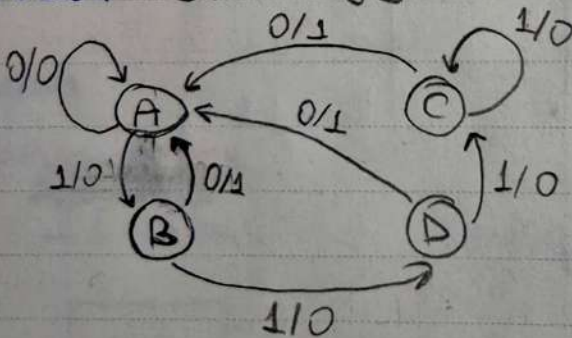
Durum Tablosu oluşturulur

$q_1 q_2$	$Q_1 Q_2$		z	
	$x=0$	$x=1$	$x=0$	$x=1$
0 0	0 0	0 1	0	0
0 1	0 0	1 1	1	0
1 0	0 0	1 0	1	0
1 1	0 0	1 0	1	0

Harf	Durum
A	00
B	01
C	10
D	11

3. adım

Durum Diyagramı oluşturulur



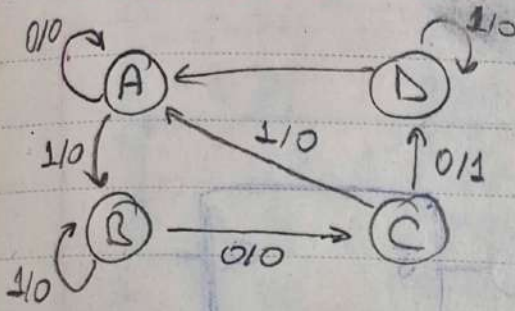
4. adım

CLK analizi yapılır.

(x değiştiği an z değişir çünkü diyagram başlı)

(Gerçekleşmesi)

Ardışıl Devre Tasarımı



Simdiki	Sonraki		Çıkış	
	x=0	x=1	x=0	x=1
A	A	B	0	0
B	C	B	0	0
C	D	A	1	0
D	A	D	0	0

(Eğer)

Adım 1) Dikey Hale Getirme (Doğruluk Tablosu)

JK için:

q Q	JK
00	0X
01	1X
10	X1
11	X0

x	q ₁ q ₂	Q ₁ Q ₂	(J ₁ K ₁)	J ₂ K ₂	z
0	(A) 00	00	0X	0X	0
0	(B) 01	10	1X	X1	0
0	(C) 10	11	X0	1X	1
0	(D) 11	00	X1	X1	0
1	(A) 00	01	0X	1X	0
1	(B) 01	01	0X	X0	0
1	(C) 10	00	X1	0X	0
1	(D) 11	11	X0	X0	0

5 adet Denklem bulacağız

J₁ K₁ J₂ K₂ z

Adım 2) Karnaugh ile denklem bulma

q ₁ q ₂	00	01	11	10
x=0	0	1	X	X
x=1	0	0	X	X

$$J_1 = x'q_2$$

q ₁ q ₂	00	01	11	10
x=0	X	X	1	1
x=1	X	X	1	1

$$K_1 = x'q_2 + xq_2' = x \oplus q_2$$

q ₁ q ₂	00	01	11	10
x=0	1	X	X	1
x=1	1	X	X	1

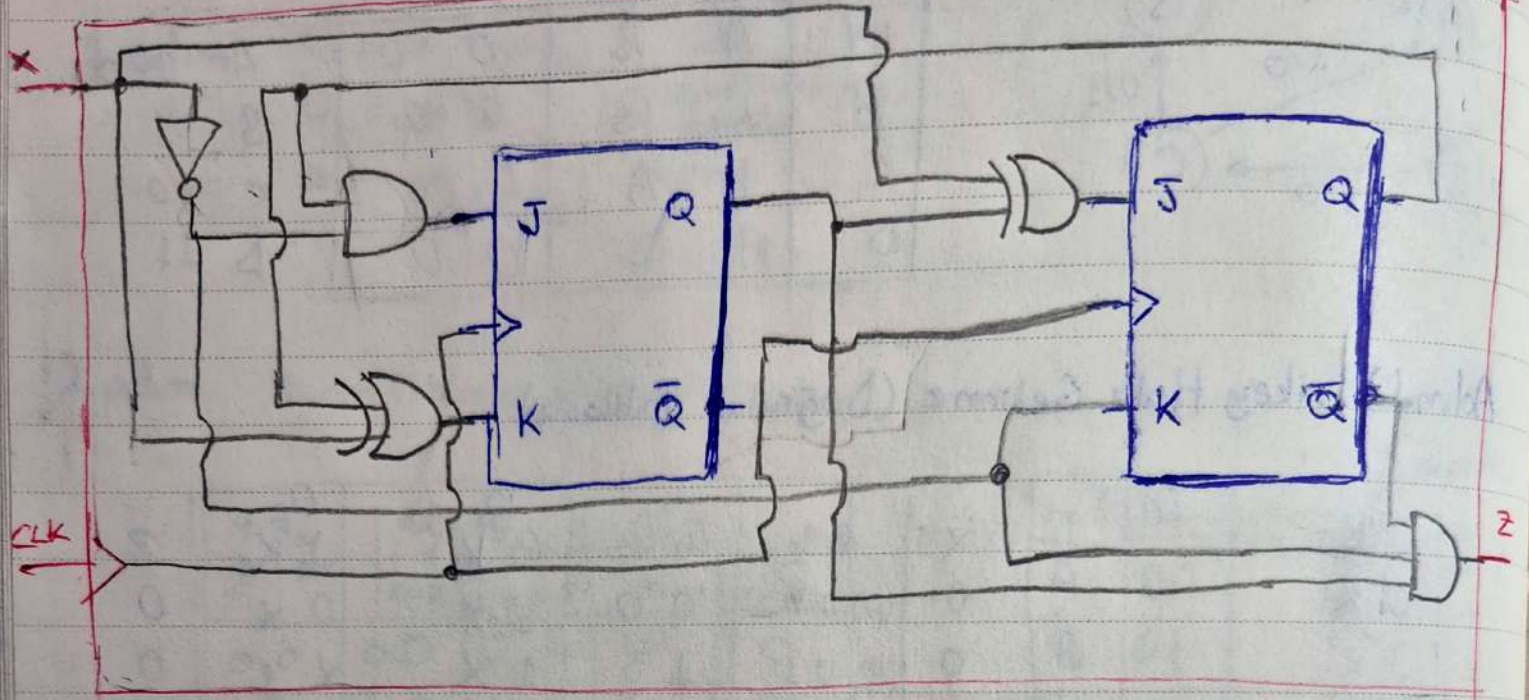
$$J_2 = x'q_1 + xq_1' = x \oplus q_1$$

q ₁ q ₂	00	01	11	10
x=0	X	1	1	X
x=1	X	1	1	X

$$K_2 = x'$$

$$z = x'q_1q_2'$$

Adım 3) Devrenin Gerçeklenmesi



$$J_1 = x'q_2 \quad K_1 = x \oplus q_2 \quad J_2 = x \oplus q_1 \quad K_2 = x' \quad Z = x'q_1q_2'$$

Senkron Devre Tasarımı Adımları (mealy)

- 1) Sözlü tanımdan, durumlar ve bellek E. hangi değerleri alacağı belirlenir.
- 2) Durum Tablosu ve diagramı
- 3) Flip Flop seçimi ve uyarma tablosu oluşturulması.
- 4) Lojik ifadelerin eldesi.

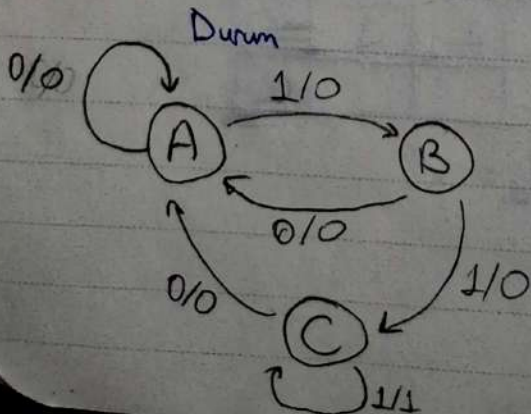
Örnek: Son 3 girişi 1 olduğunda 1 çıkışını veren Mealy tipli devre tasarlayın. JK kullanılacak

x: 0110101110 111101
z: 000000001 000110

A: Başlangıç Durumu (Hic 1 gelmedi)

B: Başlangıç +1

C: Son 2 giriş 1



x	$q_1 q_2$	$Q_1 Q_2$	J_1	J_2	K_1	K_2	z	Uygulama Tablosu
0	A 00	A 00						$q Q J K$
0	B 01	A 00	0x	0x	0			00 0x
0	C 10	A 00	0x	x1	0			01 1x
0	D 11	-xx	x1	0x	0			10 x1
1	A 00	B 01	xx	xx	x			11 x0
1	B 01	C 10	0x	1x	0			
1	C 10	C 10	1x	x1	0			
1	D 11	-xx	x0	0x	1			
1	D 11	-xx	xx	xx	x			

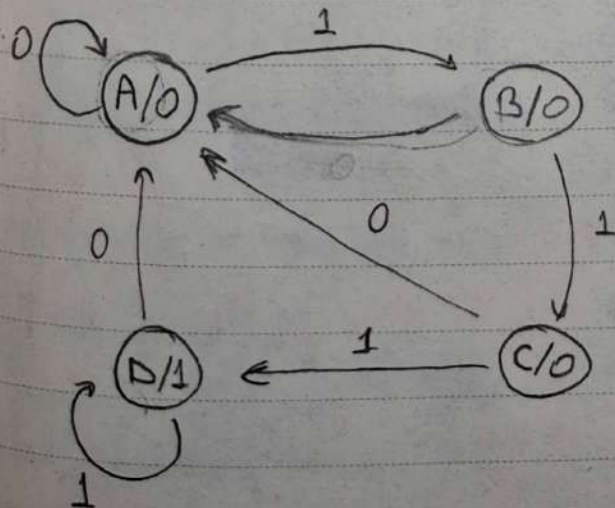
$$J_1 = x \cdot q_2 \quad J_2 = x \cdot q_1' \quad K_1 = x' \quad K_2 = 1 \quad z = x \cdot q_1$$

mealy tipi devrelerde giriş senkron değil x ile beraber giriş de değişir.
C durumuna gelince $x=0$ 'sa $z=0$ $x=1$ 'se $z=1$ olur.

Senkron Devre Tasarımı Adımları (Moore)

Moore tipinde girişler çıkışa doğrudan bağlı değildir. Dolayısıyla çıkış CLK sinyaliyle değişir.
Çıkış sadece flip flopların o anki durumuna bağlıdır.

Aynı Soruyu moore için ele alalım



A: Başlangıç (Hic 1 yok)

B: Başlangıç + 1

C: Başlangıç + 2

D: Son 3 giriş 1

x: 0 1 0 1 1 0 1 1 1 0

Durum: A A B A B C A B C D A

z: 0 0 0 0 0 0 0 0 1 0

Sadece buraya başlı
çıkış

x	q_1, q_2	Q_1, Q_2	J_1, K_1	J_2, K_2	z	q_1, q_2	J, K
0	A 00	A 00	0x	0x	0	00	0x
0	B 01	A 00	0x	x1	0	01	1x
0	C 10	A 00	x1	0x	0	10	x1
0	D 11	A 00	x1	x1	1	11	x0
1	A 00	B 01	0x	1x	0		
1	B 01	C 10	1x	x1	0		
1	C 10	D 11	x0	1x	0		
1	D 11	D 11	x0	x0	1		

J_1 için

	00	01	11	10
0	0		x	x
1		1	x	x

$J_1 = x \cdot q_2$

K_1 için

	00	01	11	10
0	x	x	1	1
1	x	x		

$K_1 = x'$

J_2 için

	00	01	11	10
0	0	x	x	0
1	1	x	x	1

$J_2 = x$

K_2 için

	00	01	11	10
0	x	1	1	x
1	x	1	0	x

$K_2 = x' + q_1$

z için

	00	01	11	10
0			1	
1			1	

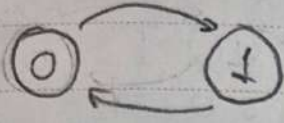
$z = q_1 \cdot q_2$

$\rightarrow$ x'den bağımsız
state senkron

Senkron Sayıcı Tasarımı:

Sayıclarda Giriş olmaz, çıkışı da yemeye gerek yok zaten adanlar çıkış.

1 Bitlik:



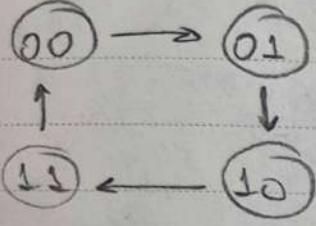
q	Q	J K
0	1	1x
1	0	x1

$$J = K = 1$$

Not: Hiç 0 yoksa 5V direkt

q	Q	J K
0	0	0x
0	1	1x
1	0	x1
1	1	x0

2 Bitlik:



q ₁ q ₀	Q ₁ Q ₀	J ₁ K ₁	J ₀ K ₀
0 0	0 1	0x	1x
0 1	1 0	1x	x1
1 0	1 1	x0	1x
1 1	0 0	x1	x1

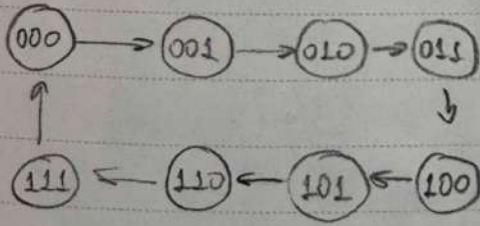
q ₁ \ q ₀	0	1
0		1
1	x	x

$J_1 = q_0$

q ₁ \ q ₀	0	1
0	x	x
1		1

$K_1 = q_0$ $J_0 = 1$ $K_0 = 1$

3 Bitlik



q ₂ q ₁ q ₀	Q ₂ Q ₁ Q ₀	J ₂ K ₂	J ₁ K ₁	J ₀ K ₀
0 0 0	0 0 1	0x	0x	1x
0 0 1	0 1 0	0x	1x	x1
0 1 0	0 1 1	0x	x0	1x
0 1 1	1 0 0	1x	x1	x1
1 0 0	1 0 1	x0	0x	1x
1 0 1	1 1 0	x0	1x	x1
1 1 0	1 1 1	x0	x0	1x
1 1 1	0 0 0	x1	x1	x1

Son 2 F/F'nin denklemleri aynı

q ₂ \ q ₁ q ₀	00	01	11	10
0			1	
1	x	x	x	x

$J_2 = q_1 q_0$

q ₂ \ q ₁ q ₀	00	01	11	10
0	x	x	x	x
1			1	

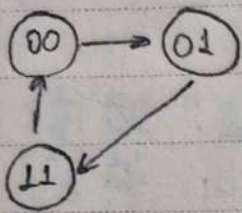
$K_2 = q_1 q_0$

Ters Sayıcı:

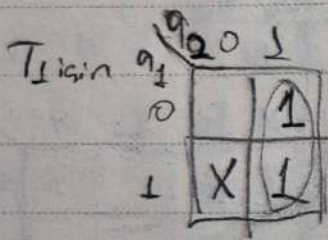
ileri sayan sayıcıda Q yerine $\bar{Q}$ 'den alınrsa geri sayan sayıcı olur.

Sıralı Olmayan Sayıcı

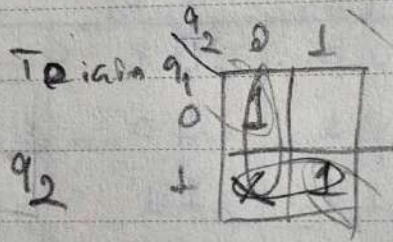
0-1-3 sayıcısını T Flip Flop'la gerçekleştirm.



$q_1 q_2$	$Q_1 Q_2$	T_1	T_0	q	Q	T
00	01	0	1	0	0	0
01	11	1	0	0	1	1
10	-XX	X	X	1	0	1
11	00	1	1	1	1	0

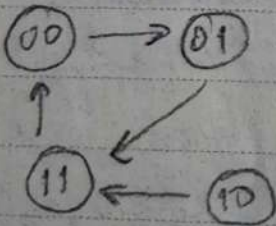


$$T_1 = q_2$$



$$T_0 = q_1 + q_2'$$

X'leri 0'mı yoksa 1'ini saydığımızı göre kararsız durumlarda nasıl davranacağını anlayabiliriz yukarıdaki devre aslında şuna eşittir.

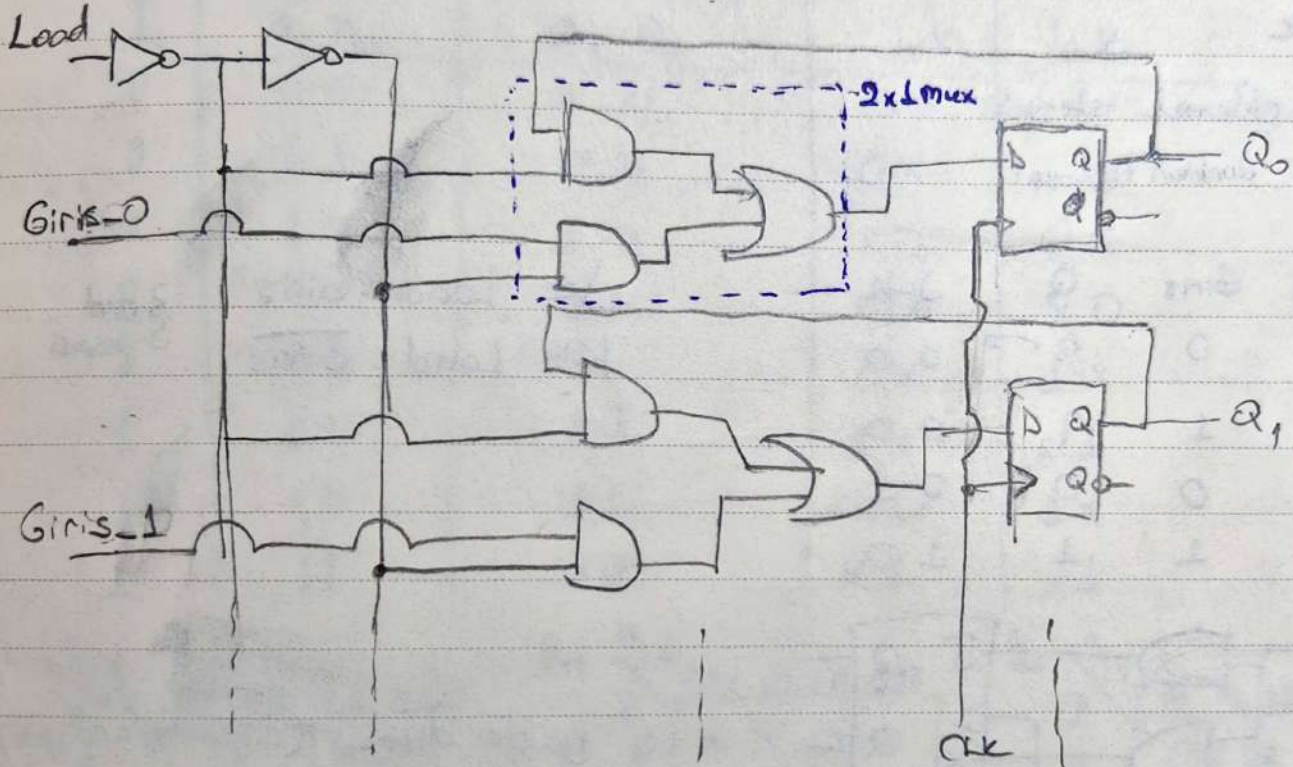


Registerler:

n adet flip-flopun bir araya gelmesiyle oluşan birimlerdir.
n adet flip-flop, n bitlik bilgiyi saklayabilir.

Paralel Değer Yükleme:

4 bitlik bir veri varsa 4 tane kabloyla aynı anda veri girisi olur. Tek CLK pulse
Giris kontrolü için Load girişi kullanılır.

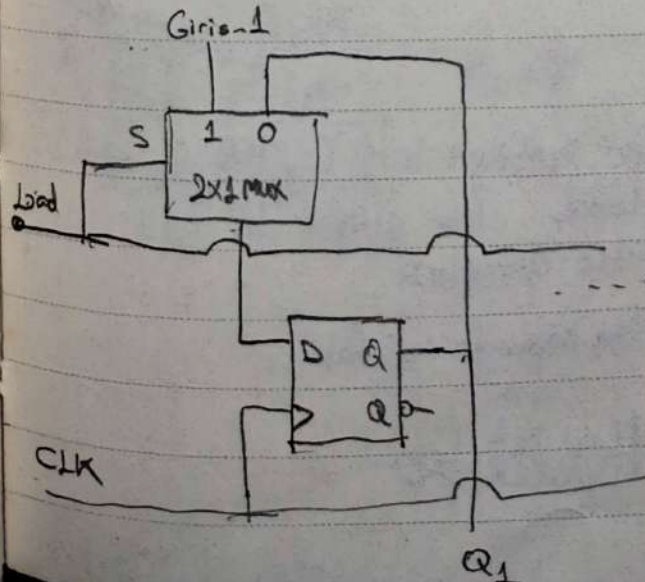


$D = (\text{Load})' \cdot Q + (\text{Load} \cdot \text{Giris})$ denklemine göre devre oluşturulur.

Load=0 bekleir Load=1 veriyi güncle

2x1'lik muxlar da kullanılabilir.

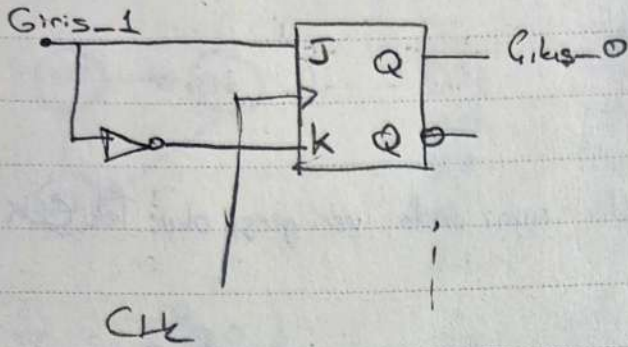
$$\text{mux} = S' \cdot I_0 + S \cdot I_1$$



→ JK'ya Paralel Değer Yüklene:

JK'nın J ucuna Giriş K ucuna $\overline{\text{Giriş}}$ bağlarsak bir D Tipi F/F elde ederiz.

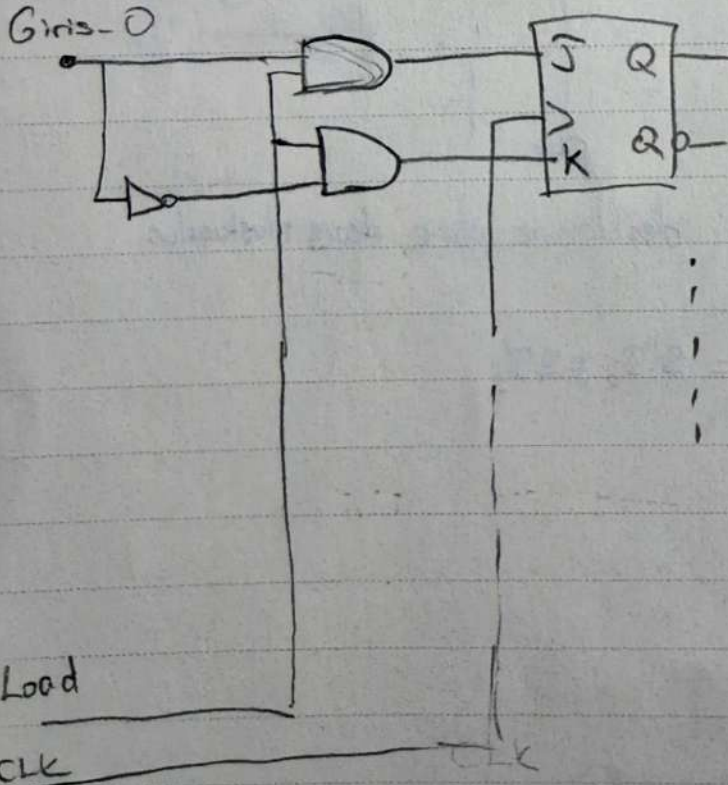
$$Jq' + K'q \rightarrow Gq' + \overline{G}q \rightarrow Gq' + Gq \rightarrow G(q' + q) \rightarrow G$$



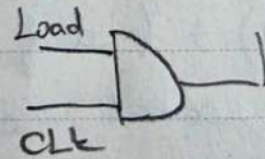
★ Load girişi eklemek istersek:
istediğimiz durumu tablosuz:

Load	Giris	Q	J K
0	0	$q \rightarrow$	0 0
0	1	q	0 0
1	0	0	0 1
1	1	1	1 0

$$\begin{aligned} J &= \text{Load} \cdot \text{Giris} \\ K &= \text{Load} \cdot \overline{\text{Giris}} \end{aligned} \quad \left. \begin{array}{l} \\ \end{array} \right\} \text{and kapısı}$$



yada direkt (diğerlerine and yok)



Eğer senkron bir CLEAR yapmak istersen clear girişini de göz önüne almalıyız.

veya kapağıyla yapılabilir

HOGPDR başı

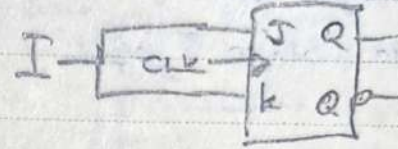
→ Register in 1 artırılması (Sayma girişi olan sayıcı)

1 Bitlik

I	Q	J	K
0	q	0	0
1	q'	1	1

$$J=I \quad K=I$$

1 1 durumu q'
0 0 durumu q



2 Bitlik

I	(simdiki d.) q ₁ q ₀	Sonraki Durum Q ₁ Q ₀	J ₁ K ₁	J ₀ K ₀
0	0 0	0 0	0 x	0 x
0	0 1	0 1	0 x	x 0
0	1 0	1 0	x 0	0 x
0	1 1	1 1	x 0	x 0
1	0 0	0 1	0 x	1 x
1	0 1	1 0	1 x	x 1
1	1 0	1 1	x 0	1 x
1	1 1	0 0	x 1	x 1

q ₁ q ₀	J ₁ K ₁
0 0	0 x
0 1	1 x
1 0	x 1
1 1	x 0

J₁ = I Q₀

I	q ₁ q ₀	00	01	11	10
0				x	x
1			1	x	x

K₁ = I q₀

I	q ₁ q ₀	00	01	11	10
0		x	x	0	0
1		x	x	1	0

J₀ = I

I	q ₁ q ₀	00	01	11	10
0		0	x	x	0
1		1	x	x	1

K₀ = I

I	q ₁ q ₀	00	01	11	10
0		x	0	0	x
1		x	1	1	x

Buna göre derre tasarımı yapılır.

Kaydırma İşlemleri:

Sıfır ile Kaydırma:

MSB yada LSB'ye 0 yazılır.

Sağa: $I_i \leftarrow I_{i+1} \quad (I_{n-1}, I_{n-2}, \dots, I_1, I_0)$

1101 $\rightarrow$ 0110

Sola: $I_i \leftarrow I_{i-1} \quad (I_{n-1}, I_{n-2}, \dots, I_1, I_0)$

1101 $\rightarrow$ 1010

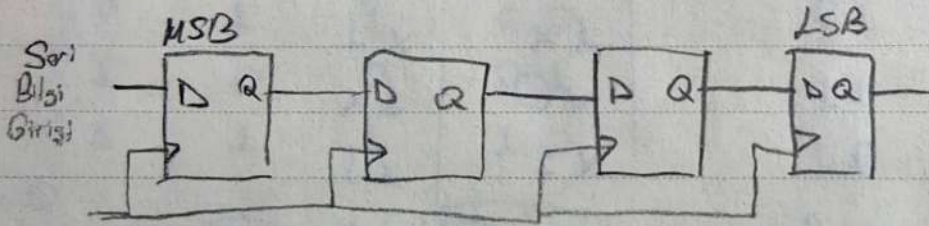
Döngüsel Kaydırma:

Sağa: $I_{n-1} \leftarrow I_0$

n bitlik register n kez kaydırılırsa başlangıca döner.

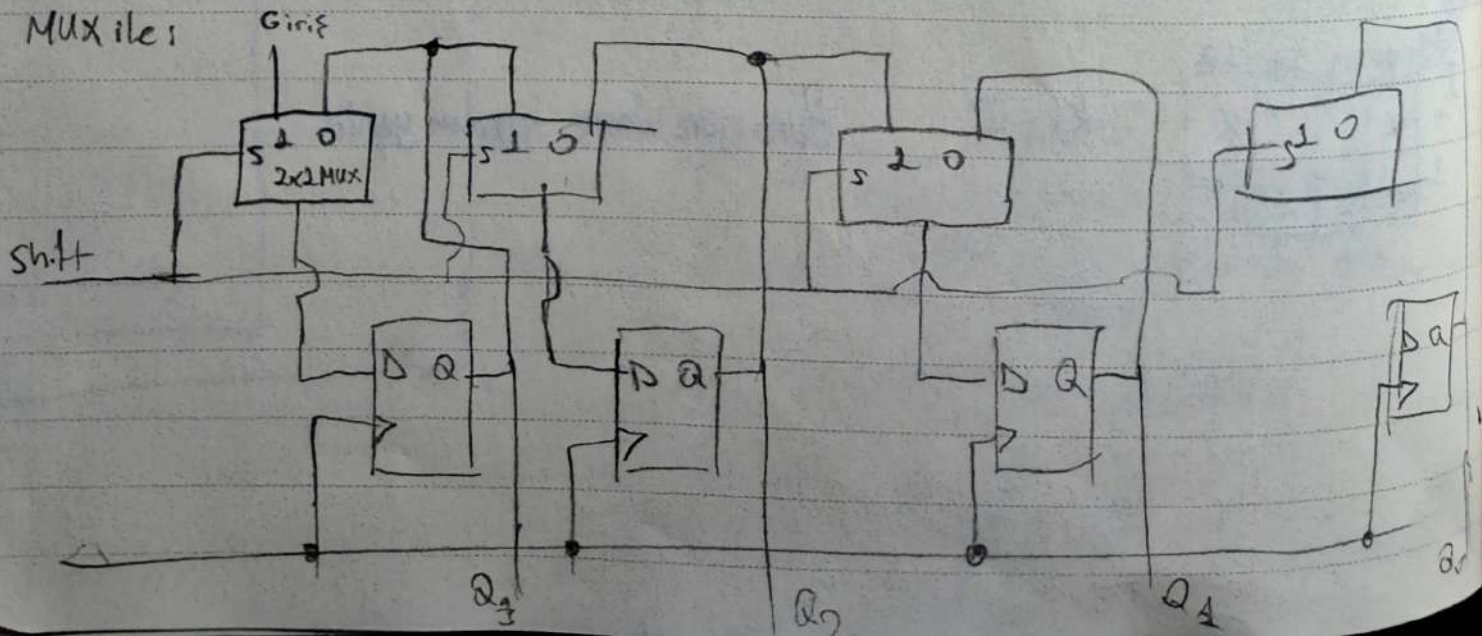
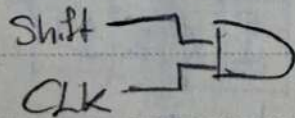
Sola: $I_0 \leftarrow I_{n-1}$

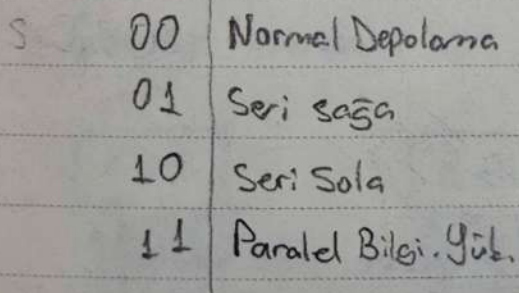
Seri Bilgi Yüklene (Kaydırarak)



Sağa ise MSB'ye
yüklenir LSB kaybolur
Solda tam tersi

Kontrollü:





Bölüm 4 Hiyerarşik Yapı

7 seviyeden oluşur

Seviye 6	Kullanıcı Seviyesi	Geliştirilebilir Programlar
5	Yüksek S. Diller	C, Java vb.
4	Assembly	Assembly kodu
3	Sistem Yazılımları	İşletim Sistemi, Derleyiciler
2	Makine Dili	Komut seti mimarisi
1	Kontrol	Donanımsal ya da mikro programlanmış
0	Lojik Seviye	Kapılar, Devreler

Assembly'de tam olarak ne kadar süre alacağı bellidir.

Mikro ders assembly 3, 4. seviye

Bil Org: saf assembly + makine 2, 1 ve 0 seviyesi.

RISC → Değişken uzun. mimarileri

CISC → sabit uzun.

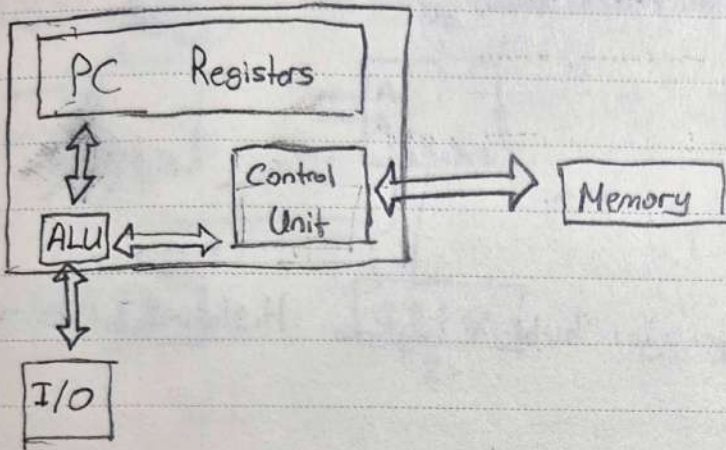
Derste RISC anlatılacak, Donanımsal kontrol anlatılacak, (günümüzde mikro kullanılır)
↓
Hardwired

von-Neumann Mimarisi

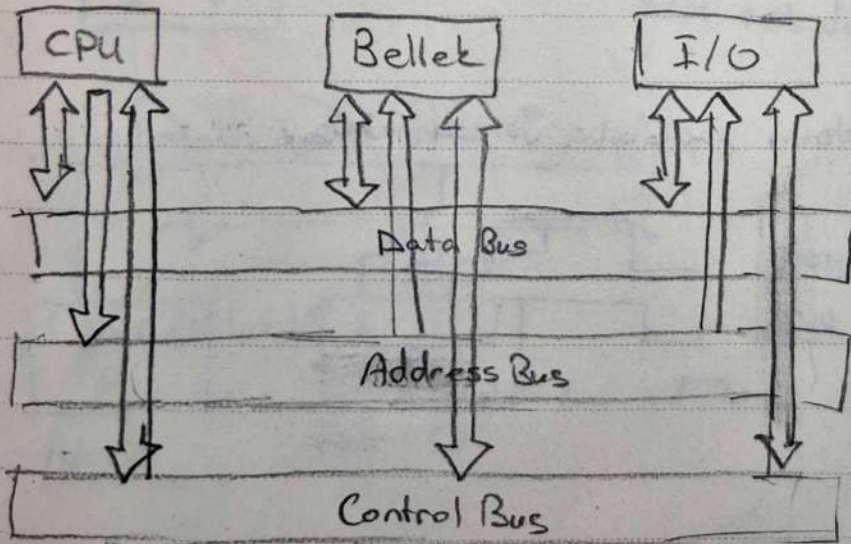
3 Temel bileşenden oluşur. CPU, bellek, Giriş/Gıkış cihazları.

CPU: ALU + kontrol ünitesi + P.C. + Other registers

P.C: bir sonraki komutu gösterir



Günümüz mimarisi için genişletilmiştir.



BUS (Yol Yapısı)

3 Tip:

→ Address

→ Data

→ Control

2 organize geçidi:

→ Tekli (Adres ve data aynı hat)

→ Çoklu (Adres ve data ayrı hat)

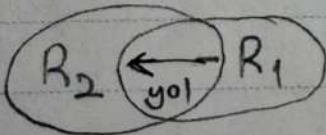
(mikro dersinde çoklu)

Bağlantı Türleri:

→ Birebir Bağlantı: Her register ayrı ayrı birbirine bağlıdır. Hızlıdır fakat karmaşık.

→ Ortak Bus: Aynı anda 1 aktarm yapılabilir. Yavaşdır. Dolaylı aktarm yapılır.

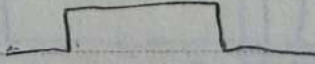
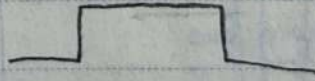
$$R_2 = R_1$$



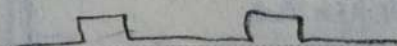
R_1 'den yol

yoldan R_2 'ye

CLK



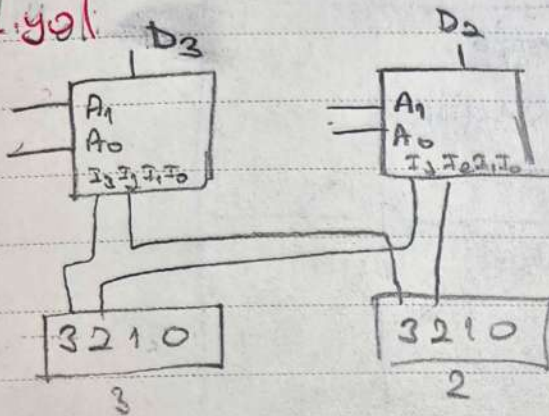
(Load işlemi var)



1. MUX kullanmak

2. Uzun duranlık buffer (tris tate) kullanmak

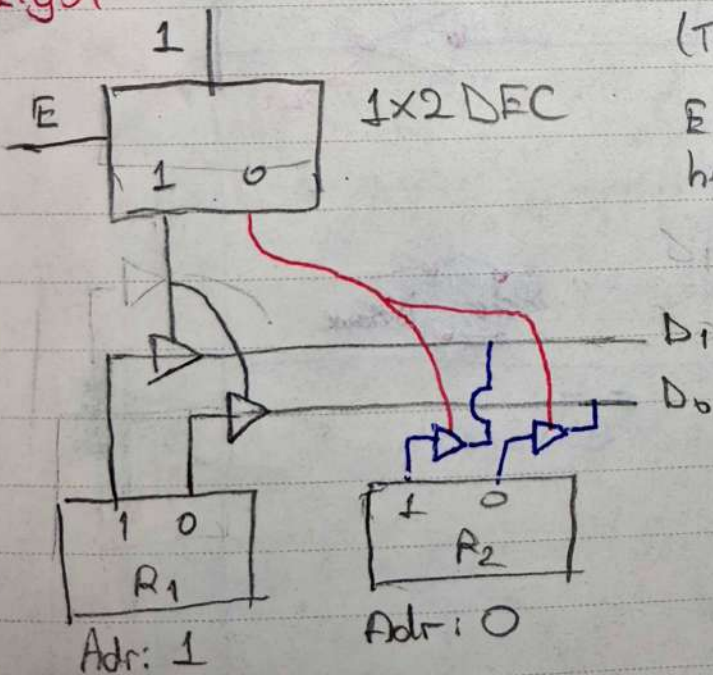
1. yal.



0 gönderme ve hissi gönderme
Sikliği.

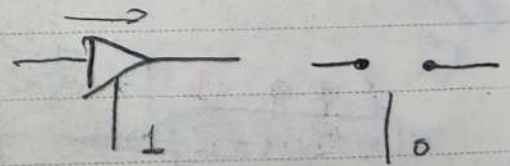
MUX'lara sinyal gelmediğinde 0
dur ve seni 0 gelmiş gibi dur.

2. yol

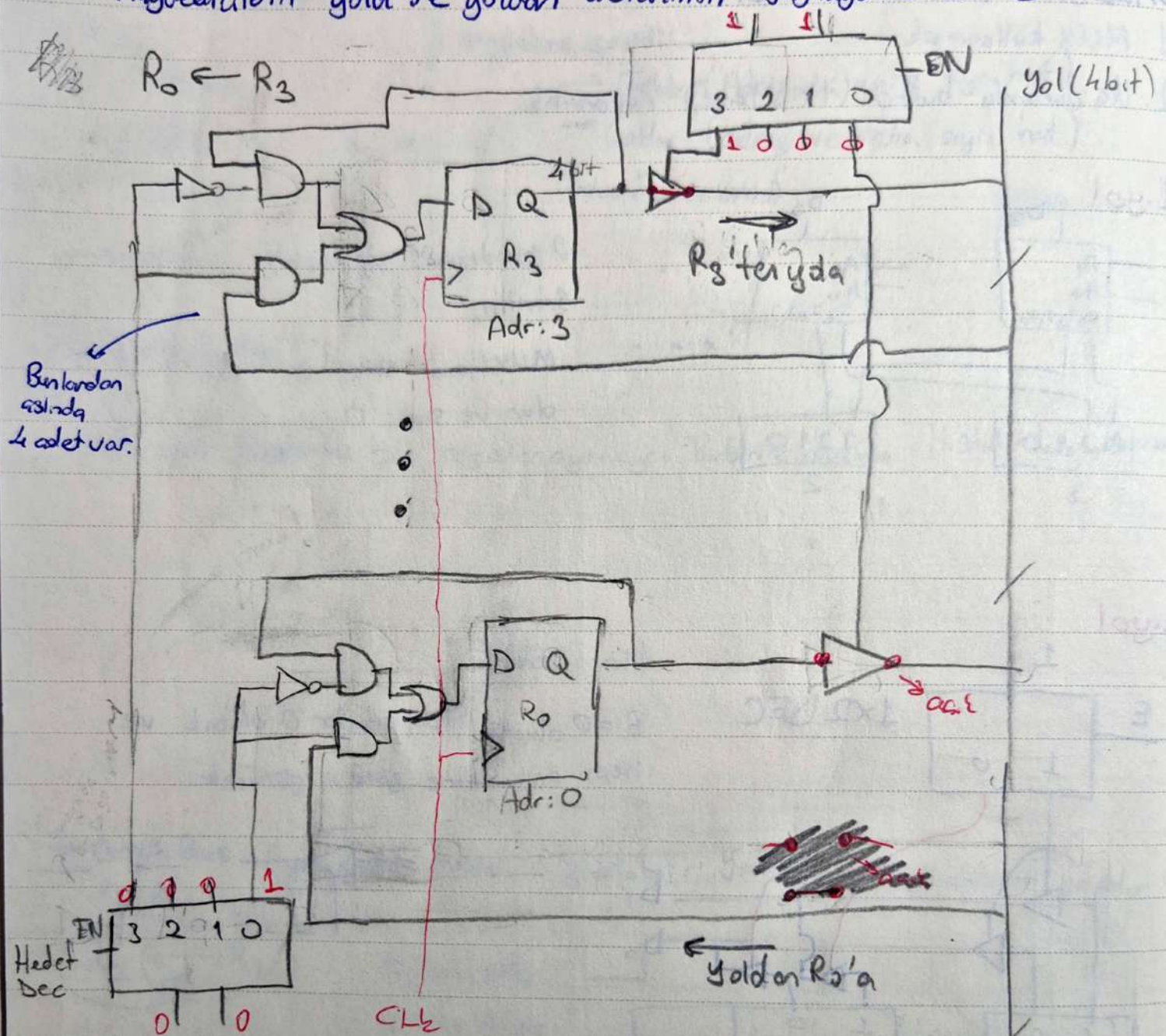


(Tam Şena)

$E=0$ olursa tüm kişiler 0 olacak ve hepsi aynı derece görevi görecek.



Kaydedicilerin yola ve goldan aktarımını sağlayan düzere:



→ Bu derlemleri
Load sirisi oktilkes
G Veri yollarindiy e

$$D = L' \cdot q + L \cdot G$$

Hedet Dec

(Astinda Lie)
tone vor

Soru Örnekleri (Vize)

$s = 1$ iken, en anlamlı bit Q_2 ise "0 ile sağa kaydırma"
 " 1 ise "0 ile sola kaydırma"

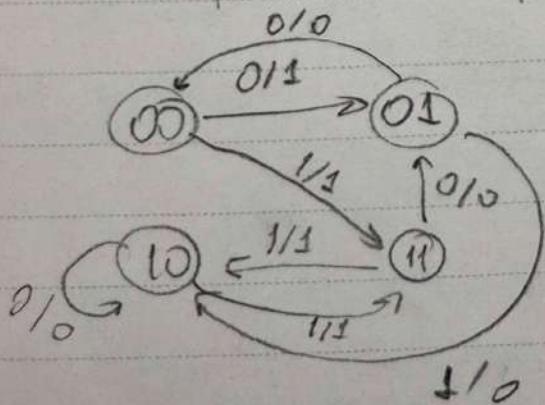
T tipi, T_2 'nin uyarma işlevi nedir

q_2	q_1	q_0	$Q_2 Q_1 Q_0$		$T_2 T_1 T_0$		
			$s=0$	$s=1$	$s=0$	$s=1$	
0	0	0	000	000	000	000	
0	0	1	001	000	000	001	
0	1	1	011	001	000	010	
0	1	0	010	001	000	011	
1	1	0	110	100	000	010	
1	1	1	111	110	000	001	
1	0	1	101	010	000	111	
1	0	0	100	000	000	100	

$$T_2 = S \cdot q_2 \cdot q_1$$

çiz dönüşüm

$q_1 q_0$	$Q_1 Q_2$		$S_1 R_1$		T_2	z	$S = x \quad R = x' \cdot q_2$
	$x=0$	$x=1$	$x=0$	$x=1$	$x=0 \quad x=1$	$x=0 \quad x=1$	
00	01	11	00	10	11	1	$T = x + q_1'$ $z = xq_1 + q_1' \cdot q_2'$ $x=0 \Rightarrow q_1' \cdot q_2'$ $x=1 \Rightarrow q_1 + q_1' \cdot q_2'$
01	00	10	01	10	11	0	
11	01	10	01	10	01	0	
10	10	11	00	10	01	0	



2. yöntem

$$Q_1 = S_1 + R_1 q_1$$

$$T = T_2 \oplus q_2$$

$$Q_1 = S_1 + R_1' q_1$$

$$= x + (x + q_2') q_1$$

$$= x + x q_1 + q_1 q_2'$$

$$= x + q_1 q_2'$$

$$Q_2 = T_2 \oplus q_2$$

$$= (x + q_1') \oplus q_2$$

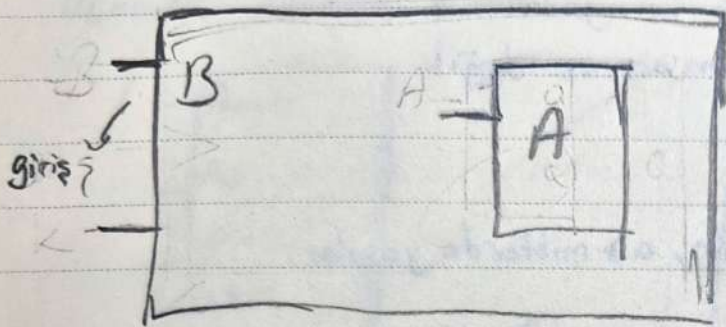
$$x=0 \text{ için } Q_2 = q_1' \oplus q_2 = q_1 \odot q_2$$

$$x=1 \text{ için } Q_2 = 1 \oplus q_2 = q_2'$$

Sınav Örneği: bir flip flopu başkasıyla gerçekleştirme

A flip flopu kullanılarak B nasıl gerçekleştirir?

Uygulama Tablosu



A'nın Uygulama Tablosu

Q	A
0	0
1	1

girişler:

B	Q	A
0		
1		
0		
1		

B için Doğruluk tablosu yazılır. A'nın uygulama T. ile A bulunur.

Amaç Q'yu ardından A'yı bulmak.

Bellek Türleri

Harddisk (elektromekanik olanlar) random access değil.

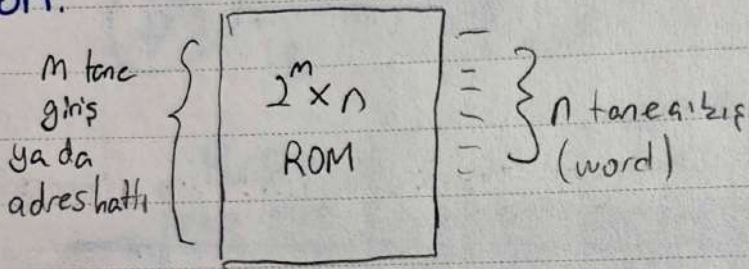
RAM $\rightarrow$ RIWM

ROM $\rightarrow$ Kalıcı bellek, genelde okunur, az miktarda yazılır.

byte $\rightarrow$ 8 bit

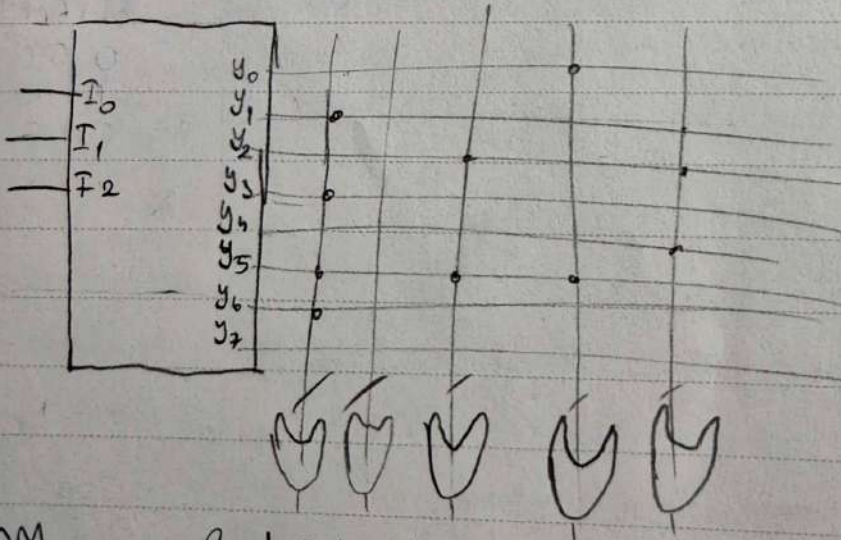
word $\rightarrow$ 1byte, 2byte, 4byte ... 8 byte ... mimariye göre değişir.

ROM:



8x8 bitlik bir ROM yapısı:

3x8'lik decoder kullanılır.



ROM

Read Only Memory

PROM

Programmable ROM

EPROM

Erasable PROM

E²PROM

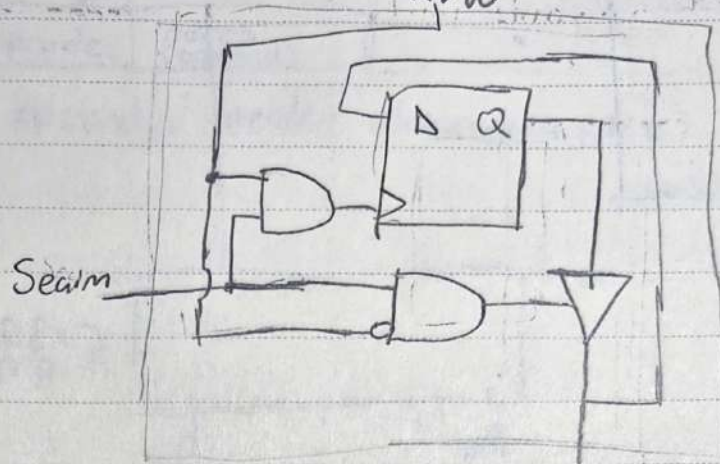
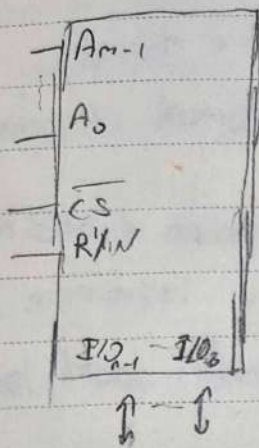
Electric EPROM (aynı anda 1 wordlük veri transferi)

FLASH

veriler blok olarak okunup yazılabilir.

RAM:

Uçucudur. Herhangi bir lokasyona erişim süresi aynıdır.



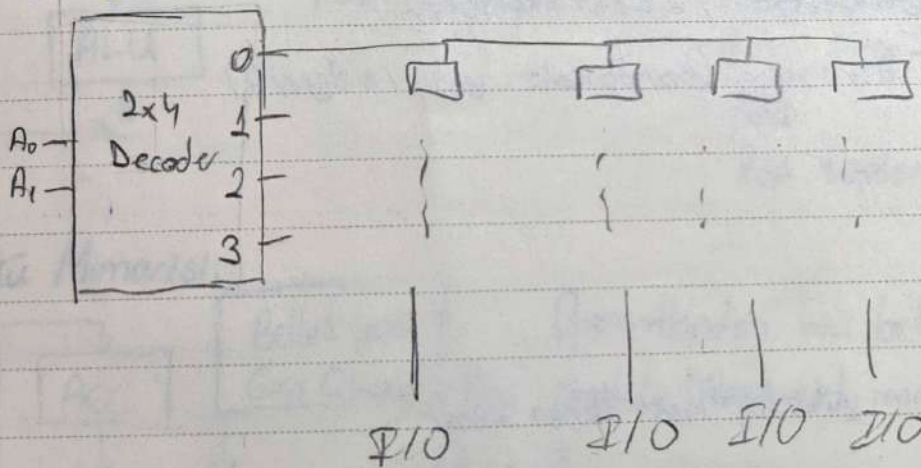
RAM CHIP:

Satır Bazlı Erişim:

4x4'lük:

Satır bazlı erişim:

10kb için 10x1024 decoder lazım



10kb x 8'lik için

1. yöntem Satır bazlı, 10x2¹⁰ decoder lazım

2. yöntem satır/sütun bazlı, iki adet 5x2⁵'lik lazım

321 nolu adres seçilirken

01010 00001

Satır Sütun

01010 00001

(10)₁₀ (1)₂

* Not: 2¹⁰ = 1kb

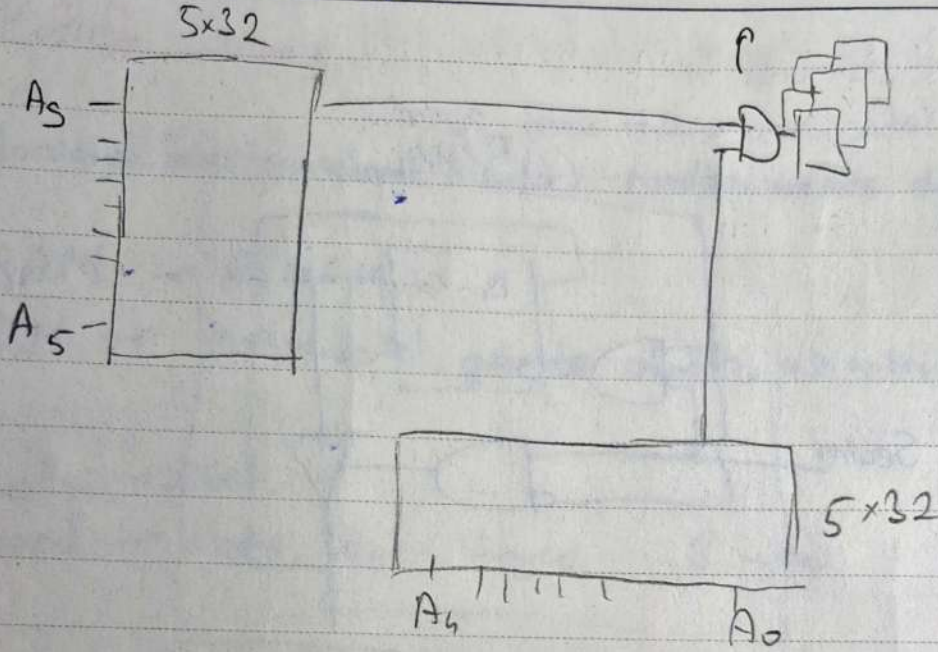
2¹¹ = 2kb

2¹² = 4kb

2¹³ = 8kb

2¹⁴ = 16kb

Satır Sütun Bazlı Erişim:



RAM Türleri:

Dinamik (DRAM) 1 adet kondansatör, 1 adet transistör

Statik (SRAM) 6 civarı transistör. (Kondansatör yok, maliyetli)

cache bellek $\rightarrow$ statik

ana bellek $\rightarrow$ dinamik

DDR (Double data Rate) hem yükselen hem düşen kenar.

SDRAM (Serik dinamik)

Bölüm 5 Komut Seti Mimarisi

Komut opcode ve adres bölümlerinden oluşur. Adresteki veriye operand denir.

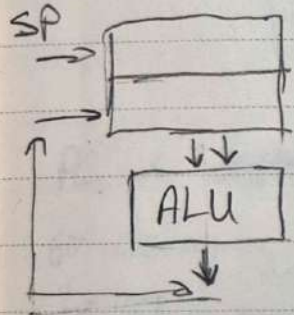
Komut =

opcode	adres (operand)
--------	-----------------

Bir işlemcinin komut seti, operandın nereden alınacağına göre;

- 1) Yığın Stack mimarisi
- 2) Akü mimarisi
- 3) Genel Amaçlı register mimarisi

Yığın Stack Mimarisi:



İşlemler stackin üst elemanlarıyla olur. Sonuç yine yığna atılır.

Push ve pop

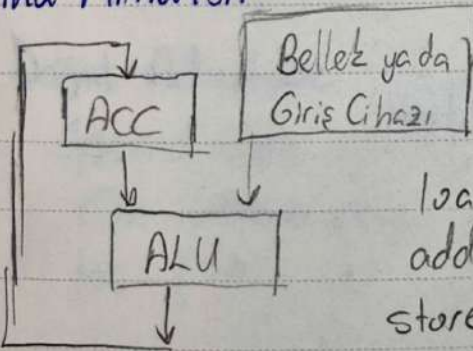
Push sayı 1

Push sayı 2

Add

Pop toplam

Akü Mimarisi:



Operandlardan biri bellekte biri aküde. Sonuç aküye yazılır. Akü imalıdır. (Yazılabilir)

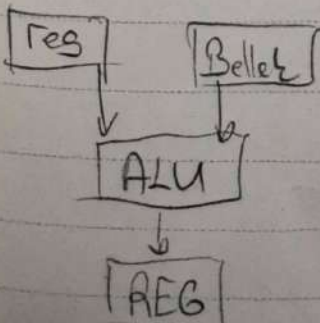
load sayı 1

add sayı 2

store toplam.

Genel Amaçlı Register Mimarisi:

Operandlar register veya bellek olarak belirtilir. Reg-Bellek yada reg-reg



load Reg1, sayı 1

Add Reg2, Reg1, sayı 2

Store Reg2, toplam

load Reg1, sayı

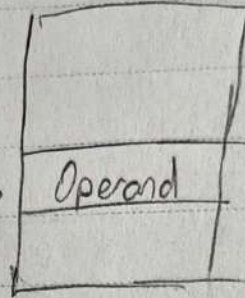
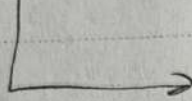
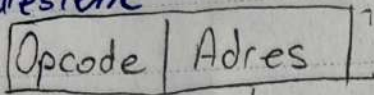
Adresleme Metotları:

İvedi Adresleme:

Opcode Operand

Add #3

Direkt Adresleme



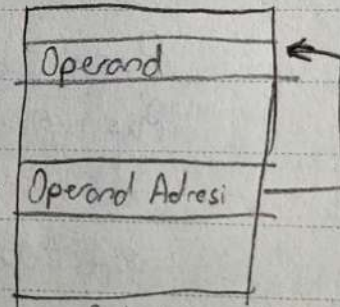
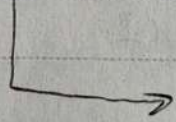
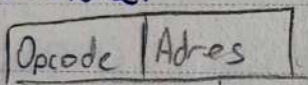
Bellek

Add 100

→ 100'in içindeki sayı

Sınırlı adres kullanımı

Dolaylı Adresleme:

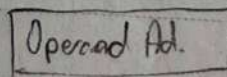
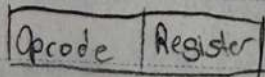


Bellek

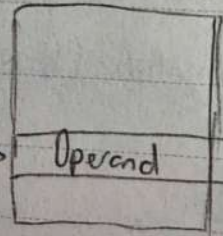
Add (100)

'()' kullanılır.

Register - Dolaylı



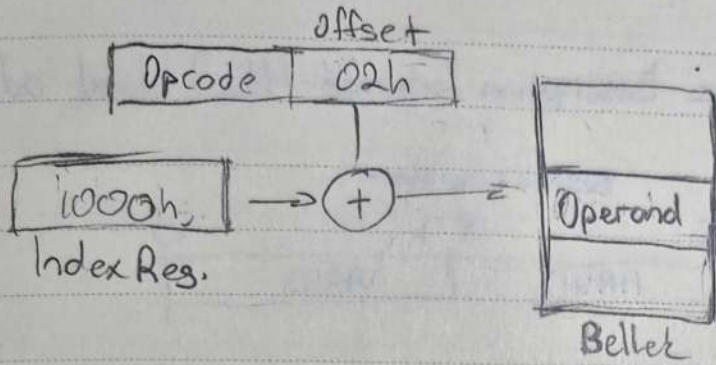
Register



Add(R1)

Adress kullanımı genişletmek mümkündür.
Direkte göre işletim zamanı daha uzun.

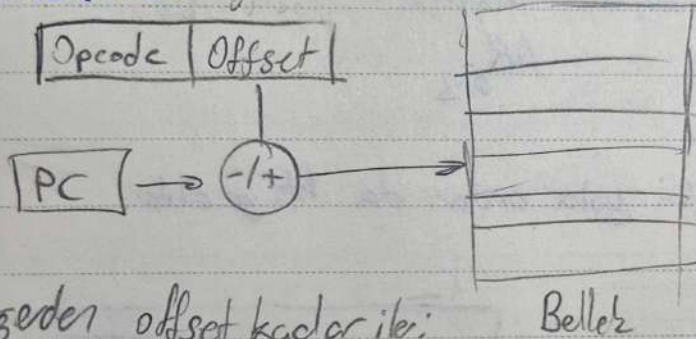
Indirs (index) Adresleme



$* \leftarrow 1000h$
Add $*02h$
Add $1002h$) aynı

Index reg'i bir kere
alınır sonra o reg'e göre
* ile değer belirlenir.

Göreceli Adresleme:



PC'deki değerden offset kadar ileri:
geri dallas

→ 80h'tan sonrası -'dir.

Doğal Adresleme (Inherent)

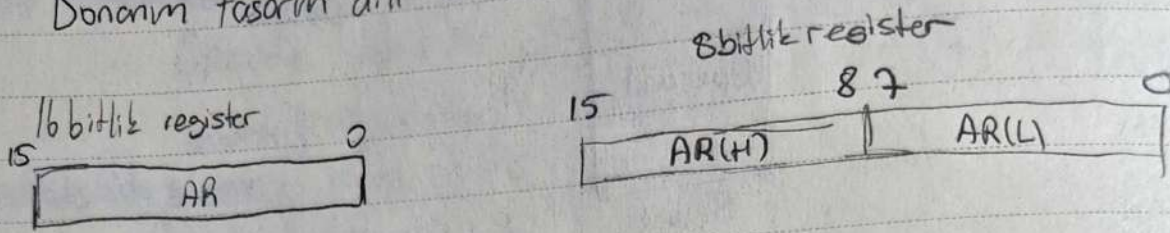
Operand bilgisi içermez.

Clr (Akünün ırgılgını temizle)

Inc ('n 1 artır)

Register Transfer Dili

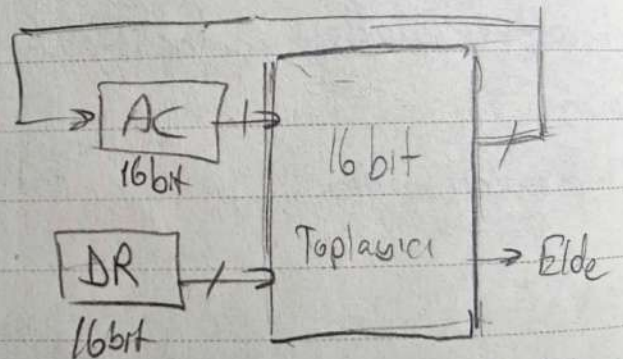
Donanım tasarım dili (Hardware Description Language - HDL) olarak adlandırılır.



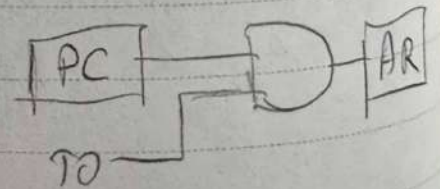
Yüksek anlamlı kısım AR(8-15) ya da AR(H)
 düşük " " AR(0-7) ya da AR(L)
 Belli bir bit grubu, AR(0-3) " AR₀₋₃

AR ← PC PC'nin içeriğini yola oradan da AR'ye aktar.

AC ← DR + AC Toplama
 AC ← DR - AC Çıkarma
 " " AC' 1'e tım.
 " " AC' + 1 2'ye tım.
 " " AC ANDR Ve
 " " AC UDR Veya
 " " SHR(AC) Sağa kaydırma
 " " SHL(AC) Sola "



TO: AR ← PC TO sırtı seçilirse yap



X: AR ← PC, PC ← PC + 1

bu
yol kullanmaz
aslında.

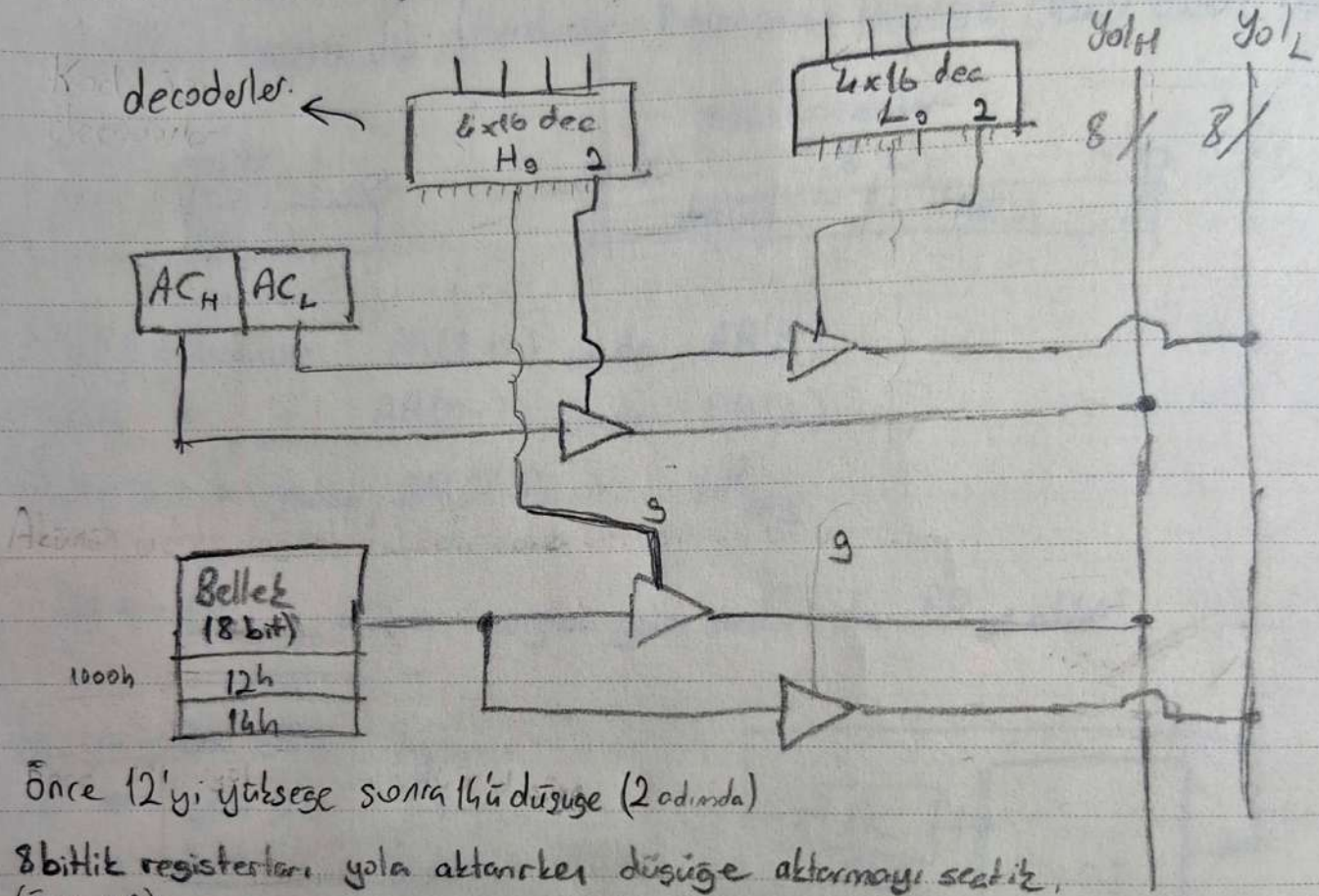
PC++

Aynı anda yapılabilir

Temel Bilgisayar Mimarisi Tasarımı:

ACC'de dec ucu da bulunur.
Bellez 2 adim da yola aktarilabilir.

Akunin İgeriginin Yola Aktarılması
(2 ile)



Durum Kod Kaydedicisi (CCR)

Belleğin yola aktardması
(9 ile)

[- - - I V N Z C] ALU ile gelir.

↙ ↘ ↓ ↓ ↓
Kesme Taşma İşaret sbr Elde

Komut Tasarımı

60 Komut ^{5 bit yaptık} ~~6 bit~~

6 adr metodu ^{3 bit}

~~5 bit~~ fakat 8 ile yaptık.

Opcode | Operand

↑
Add # 1234h

işlem tipi aynı olan birden fazla komut olabilir. Bunları ayırt ederken adresleme metodları kullanılır.

ADDC

ivedi, direkt, dolaylı, index (Doğal, göreceli, desteklenez)

Örnek

CLR

doğal

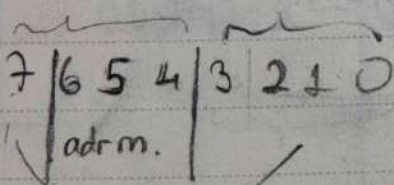
BCC

göreceli

Bu 3 komutu aynı işlem tipi gibi düşünüp adr metoduna da bakarak bu komutları ayırabiliriz. Bu sayede 5 bit yerine 8 bit kullanabiliriz.

OPCODE

Bit (654)



işlem tipi

adr met.

işlem tipi

000

Doğal Mod

001

ivedi

010

Direkt

011

Dolaylı

100

Index

101

Göreceli

0|001| 0000

10h

0|010| 0000

20h

7 3 2 1 0 → işlem tipi

6 5 4 → adr metodu

Komut incelemeesi

CLR

ADD (1000h)

BZR ~ 05h

HLT → (3415)

★ Önce yüksek sonra düşük adres
Big-endian gösterimi kullanırız.

Bu opcode'lar Komut registerına
alınır ve burada decoder'lar ile çözülür.

0000h	01h
	30h
Big endian	10h
	00h
	53h
	05h
0006h	0Eh
	...
1000h	20h
	00h
	...
2000h	12h
	34h

← PC
IR

Tsimlendirme kuralı

IRLOT3

IR registerın low output'unun 3. biti

(IR zaten 8bt)

★ Komut Çözücü

IDEC01 ve ADDRMD00

İse CLR'dır

IR Decoderi 004'tesi

İşlem Tipi 5x32

IRLOT7	2 ⁴	0
" 3	2 ³	
" 2	2 ²	
" 1	2 ¹	
" 0	2 ⁰	31

IDEC00
IDEC01

IRLOT6
IRLOT5
IRLOT4

3x8
Addr Md.

ADDRMD00

7

Örne:

IDEC00 ve ADDRMD3 ADD

" 03 " 5 BZR

" 01 ve 11 0 CLR

IDEC00. ADDRMD3:

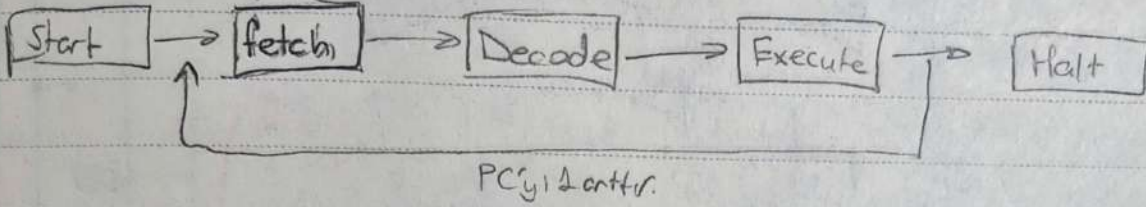
IDEC03. ADDRMD5:

IDEC01. ADDRMD0: ALCE

Komut cycle'i (Komut gevrnmi)

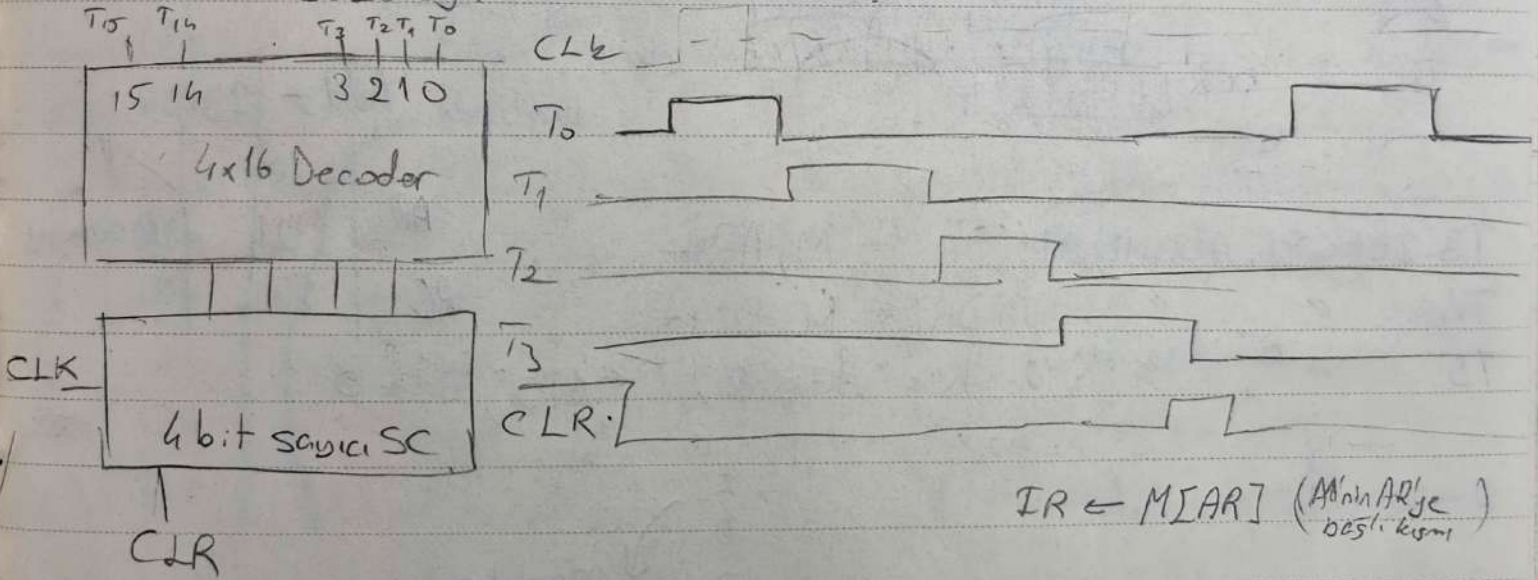
fetch
decode
execute

Execute kısımları komuttan komuta değişir.



4 bitlik bir sayıcı kullanacağız. En fazla 12 machine cycle'ında işletilen (T_{11}) bir komutumuz vardır. Bu sebeple 4 bit.

Zamanlama Düzeneği:



T0: $AR \leftarrow PC$

T1: $IR \leftarrow M[AR], PC++$, "Kod gözetme"

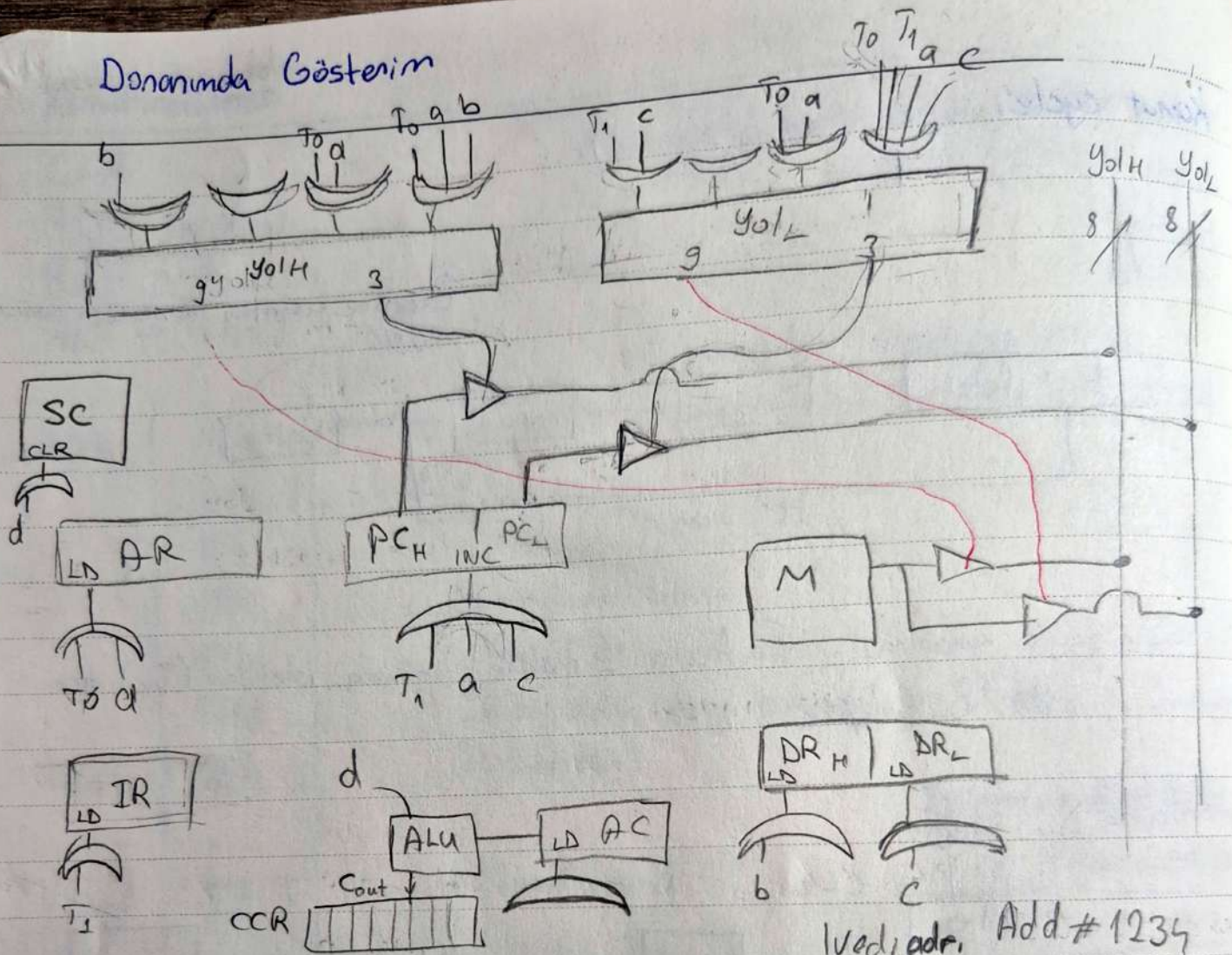
T2: ADRMDO: $AR \leftarrow PC, PC++$

Execute'a hazırlık

$PC(T0) \rightarrow Opcode \leftarrow AR$
 $PC(T1) \rightarrow \leftarrow AR[T2]$

T0 T1 T2 adımları tüm Komutlar için ortaktır.

Donanıda Gösterim



T3. IDEC00. ADDMD01: $DR_H \leftarrow M, AR++$

T4. " " : $DR_L \leftarrow M, PC++$

T5. " " : $AC \leftarrow AC + DR, C \leftarrow C_{out}, SC \leftarrow \emptyset$

İvedi adr. Add #1234

AR	10h	
	12h	1000h
PC	34h	

Execute kısmı hepsinde değişir.

Mantıksal işlemlerde elde yok

İsteriler a sermade PC'yi 2 okttu.

Direkt adresleme

Add 1000h gelirse
add direkt

PC	20h
AR	10h
PC	00h
1000h	12h
	34h

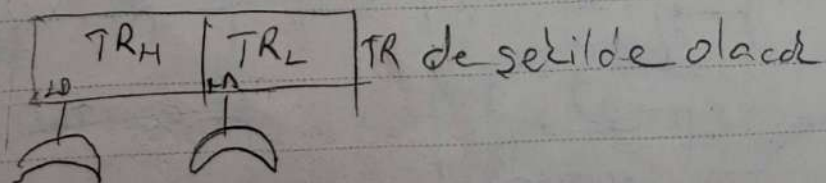
T3. IDECOO. ADRMD2: $TR_H \leftarrow M, AR++$
T4. " " : $TR_L \leftarrow M$
T5. " " : $AR \leftarrow TR$
T6. " " : $DR_H \leftarrow M, AR++$
T7. " " : $DR_L \leftarrow M, PC++$
T8. " " : $AC \leftarrow AC + DR, C \leftarrow C_{out}, SC \leftarrow 0$

Dolaylı adresleme

Add (1000h)

PC	30h
AR	10h
PC	00h
1000h	20h
	00h
2000h	12h
	34h

T3. IDECOO. ADRMD3: $TR_H \leftarrow M, AR++$
T4. " " : $TR_L \leftarrow M$
T5. " " : $AR \leftarrow TR$
T6. " " : $TR_H \leftarrow M, AR++$
T7. " " : $TR_L \leftarrow M$
T8. " " : $AR \leftarrow TR$
T9. " " : $DR_H \leftarrow M, AR++$
T10. " " : $DR_L \leftarrow M, PC++$
T11. " " : $AC \leftarrow AC + DR, C \leftarrow C_{out}, SC \leftarrow 0$



*Göreceli ve indisi adresleme metodu için;

Aynı bir adres hesaplama birimi verimliliği arttıracaktır. ALU'yu kullanmaktan vazet kaybedilir.
→ Etkin adres hesaplama birimi

Bu sayede adres hesaplanırken AC ve DR eski değerleri korun.

1010 verilerle bu birimin verisi yola aktarılır.

İndis adr.'nin offseti isaretsiz sayı

Görecelinin offseti (2'ye tımlayan) isaretili sayı.

T2 anında AR zaten offset değerini gösterir haldedir. Bellek doğrudan AR'ye bağlı olduğu için offset direkt etkin h. bir.'ne aktarılır.

→ PC ve IX 16 bittir fakat bellek yeni offset 8 bittir.

* Index adr → 8 tane 0 + offset (offsetin soluna 8 sıfır eklenir isaretsiz)

* Göreceli adr → 8 tane MEMOT + offset (bitek başlı olarak 8 0 veya 1 eklenir isaretili)

Örn: göreceli:

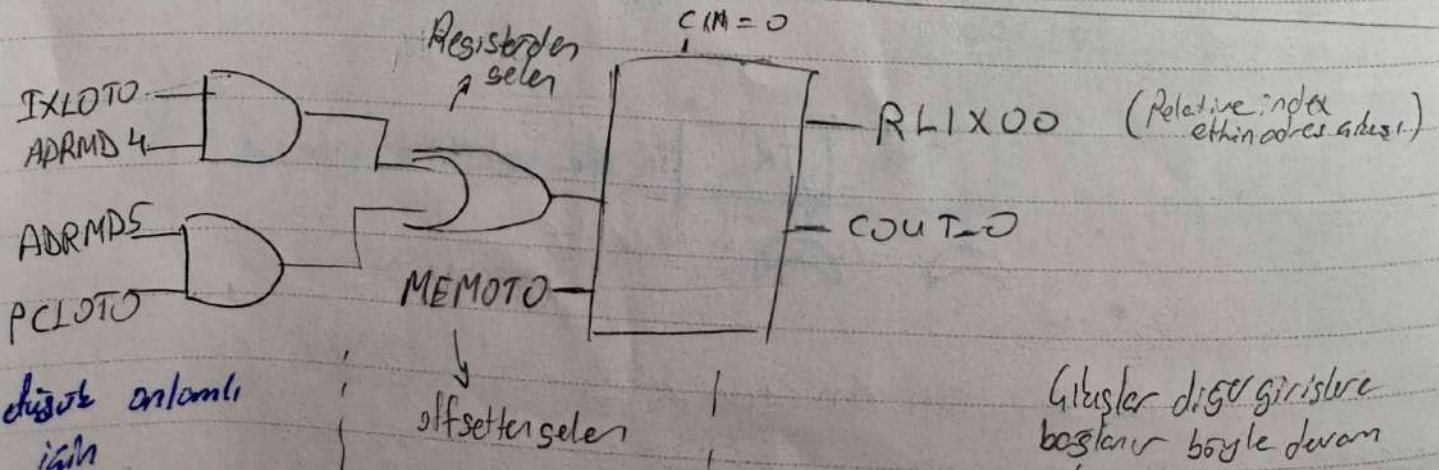
Offset: 85h = 1000 0101₂ yeni -123₁₀ genişlet → 1111 1111 1000 0101

PC = 1234h ise 1234h + FF85h = 11B9h (etkin adres)

Örn: indisi:

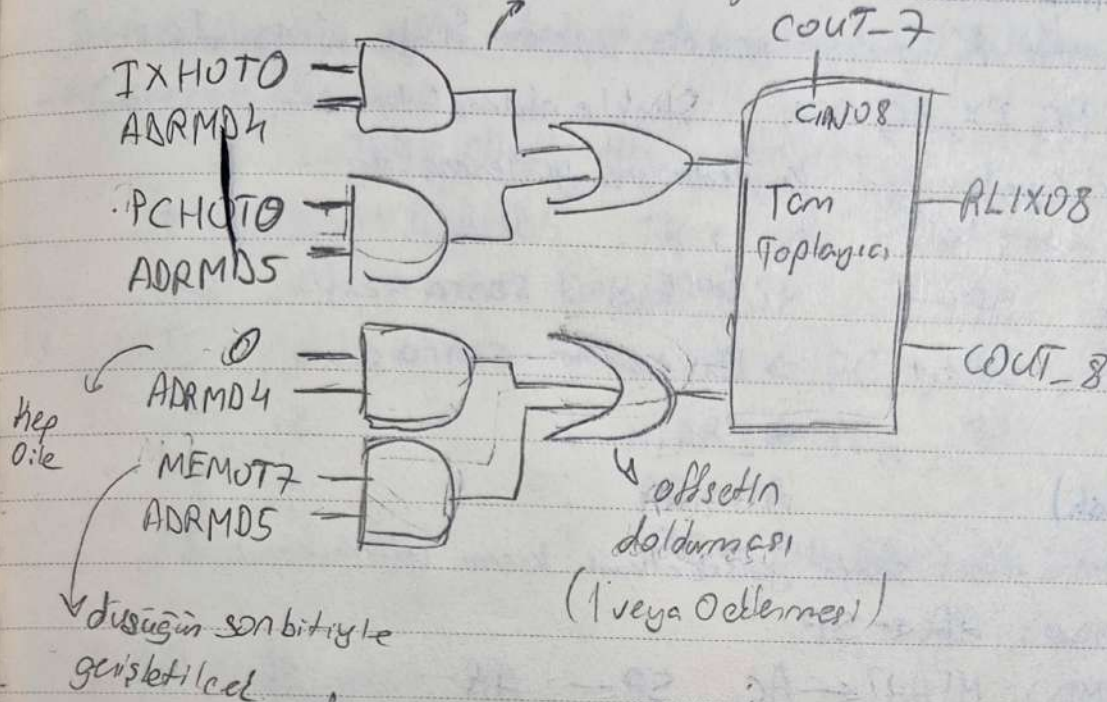
offset: 85h = 1000 0101₂ yeni 133₁₀ genişlet 0000 0000 1000 0101

PC = 1234h ise 1234h + 0085h = 12B9h



ADRMDB: gərəclı: adr. 

Registerden geleu



Örnek: Indisi ADD komutu $ADD *12h$ $01000000 = 40h$ opcode
 $\Rightarrow 12h$ offset değeridir. IX içerisine $12h$ eklenecek ve etkin adres hesaplanacak.
 $(= 1000h)$ $T2$ ve $T3$ arası IX ve $12h$ toplanır ve hesaplanır.

IX = 1000h	
PR =	40h
PC =	12h
1012h	34h
1013h	56h

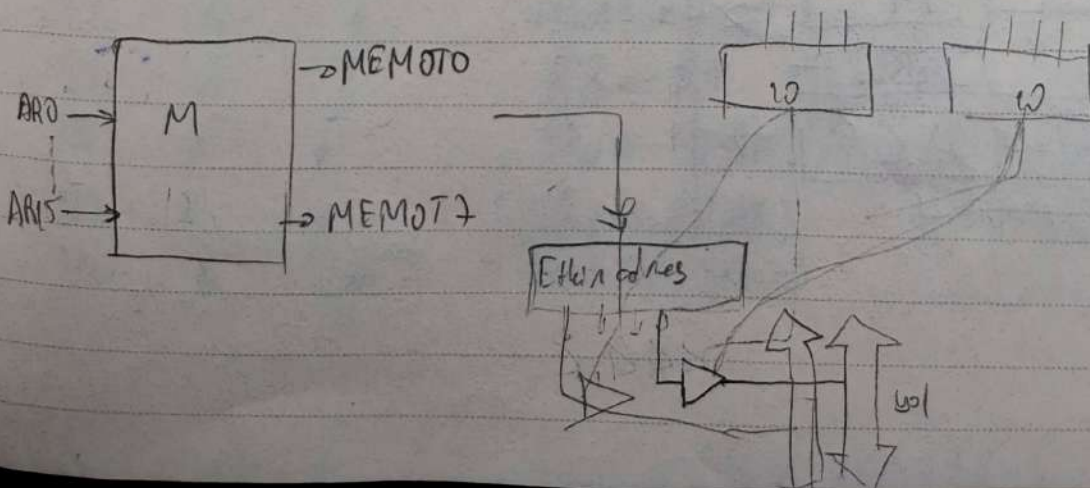
T3.IDEC00.ADRMD4: AR ← Ethen Adres

T4. IDEC00. ADAMD4: DRH ← M[AR] AR++

T5. IDECOO. ADMOD: DR, ← M[AR]

T6. IDECOO ADRMP4: $AC \leftarrow AC + DR$, $C \leftarrow C_{out}$, $SC \leftarrow 0$

T3 canında 1010 voltur (elektron adresi yola atılmamış iştir)



NOT: Stack aslında bellekte bir yedir. SP aşağıdan ver. gel. için yukarı hareket eder. Bu sebeple sayılar ters, (önce daha son. geleceklerdir).

Yığın (Stack) ve Stack Pointer

Herhangi bir dalanma ad. PC'nin değeri geri döndürülecekse SP'ye atılmalıdır.
Kesme geldiyse "PC, IX, AC, CCR" Stack'e atılmalıdır.
Kesme bitince bu değerler geri kaydedicilere yüklenmelidir.

Yığına veri eklenirse $SP--$ önce yaz sonra azalt
Yığından veri alınırse $SP++$ önce arttır sonra al

PSH Komutu: (Push)

→ Doğal adr. Akımın önce düşer sonra yükseltilmiş kısmı yığına konur.

T3. IDECO6. ADRMDO: $AR \leftarrow SP$

T4. IDECO6. ADRMDO: $M[AR] \leftarrow AC_L, SP--, AR--$

T5. IDECO6. ADRMDO: $M[AR] \leftarrow AC_H, SP--, SC \leftarrow 0$



~~PUL Komutu: (Pop)~~

→ Doğal adr: Yığındaki değer akıya konur. Yığından akıya direkt aktarım yapılır az.

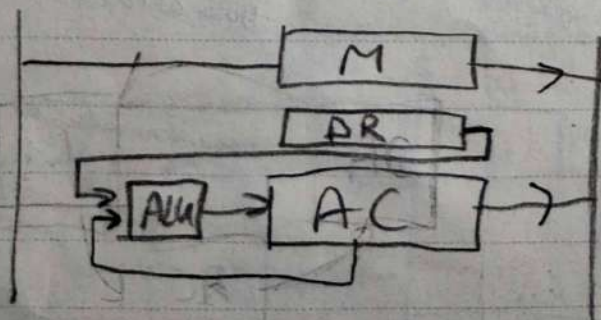
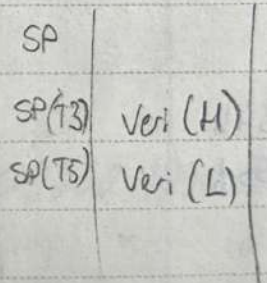
T3. IDECO7 ADRMDO: $SP++$

T4. " " : $AR \leftarrow SP$

T5 " " : $DR_H \leftarrow M[AR], AR++, SP++$

T6 " " : $DR_L \leftarrow M[AR]$

T7 " " : $AC \leftarrow DR, SC \leftarrow 0$



BSR (Branch to subroutine)

Sartlar hesaplanan etkin adresteki alt programa, kısıtlı dallanma sağlar.

→ Göreceli adresleme

T3 T2'de offset'i AR'ye atmıştık.

T3: IDEC 15. ADRMD 5: $TR \leftarrow AR$ (offset'i şimdi kiye yedekliyoruz)

T4: $AR \leftarrow SP$

T5: $MIAR] \leftarrow PC_L, AR--, SP--$

T6: $MIAR] \leftarrow PC_H, SP--$

T7: $AR \leftarrow TR$

T8: $PC \leftarrow \text{Etkin Adres}, SC \leftarrow 0$

Burda AR ve SP aynı da var. Amaç: PC'yi Stack'e yedeklemek
offset'ler geldi.

Dallanma işlemi.

BSR 005 gibi

Etkin adres hesaplanırken PC'deki değer kullanılır PC
gözetken 2h arttığından komutun opcode'ından 2 sonraki
yerden itibaren offset kodları dallanır.

Buradan devam eder.

5F 1000h
05 1001h
1002h
1007h

RTS çağırma kodları devam.

JSR (Jump to Subroutine)

→ Direkt ve indirgi dallanmayı sağlar.

Önce düşünün
palace'ın 200h PC'yi 200h
dallanacak kısmı kaybetmemek için
TR'ye atılır.

T3: IDEC 24. ADRMD 2: $TR \leftarrow AR, PC++$

T4: $AR \leftarrow SP$

T5: $MIAR] \leftarrow PC_L, AR--, SP--$

T6: $MIAR] \leftarrow PC_H, SP--$

T7: $AR \leftarrow TR$

T8: $TR_H \leftarrow MIAR], AR++$

T9: $TR_L \leftarrow MIAR]$

T10: $PC \leftarrow TR, SC \leftarrow 0$

A8
30h
00h

↑ = AR
↓ = PC ← AR

indis olsaydı

T8: $PC \leftarrow \text{Etkin adres}, SC \leftarrow 0$

RTS:

PUL'a benzer fakat sonunda AC yerine PC'ye konur.

T3. IDEC26. ADMISO: SP++

T4. // : AR ← SP

T5. // : PC_H ← M[AR], AR++, SP++

T6. // : PC_L ← M[AR], SC ← 0

Stackler alırken SP++
ve AR SP gibi davranır. AR ile bellekteki PC'ye yönlendir.

Örnek 1:

a = 5;

for i = 1 to 10 { a = a + i }

Değişkenler

0000h LDA #0005h

0003h STA 0030h ; a değeri 16 bit old için 0030 ve 0031h'ya yazılır

0006h LDA #0001h

0009h STA 0032h

000Ch LDA #000Ah

000Fh STA 0034h

0012h LDA 0030h ; a değerini alıyoruz

0015h ADD 0032h

0018h STA 0030h

001Bh LDA 0032h ; i değeri AC'ye yükleniyor

001Eh SUB 0034h ; im - i işlemi (0034'teki değ - AC)

0021h BZR ~09h ; 002Ch'a dallanır.

0023h LDA 0032h ;

0026h INCR ; i'yi 1 artırır.

0029h STA 0032h ;

002Ah BRA ~E6h ; 0012h'a git

002Ch LDA 0030h ; toplamı (a'ya) AC'ye yolla

002Fh HLT

0030h	a _H
0031h	a _L
0032h	i _H
0033h	i _L
0034h	im _H
0035h	im _L

★ offset + PC = Etkin Adr.

$$EA - PC = \text{offset}$$

$$0012h - 002Ch$$

$$0012h + (-002Ch)$$

$$0012h + FFD4 = FFE6$$

offset

★ hedef hesabı

$$0009h + 0023h = 002Ch$$

$$\text{offset} + PC = EA$$